Relazione Progetto d'esame
Programmazione e Modellazione a Oggetti

Sessione Autunnale 2025/2026

Ultima Carta

AUTORI

Andrea Rossi
Matricola: 327661

Daniele Tontini
Matricola: 328467

Indice

1 Analisi **4**

1.1 Requisiti . . . 5

1.1.1 Requisiti funzionali . . . 5

1.1.2 Requisiti non funzionali . . . 5

1.1.3 Diagramma dei Casi d'Uso . . . 6

1.2 Analisi e Modello del dominio . . . 7

2 Design **9**

2.1 Architettura . . . 9

2.1.1 Model . . . 10

2.1.2 View . . . 10

2.1.3 Controller . . . 10

2.2 Design dettagliato . . . 11

2.2.1 Rossi . . . 11

2.2.2 Tontini . . . 19

3 Sviluppo **23**

3.1 Testing . . . 23

3.1.1 ClassicRulesTest . . . 24

3.1.2 NoMercyRulesTest . . . 26

3.1.3 PointsGameTest . . . 27

3.1.4 FullGameTest . . . 28

3.1.5 CpuPlayerTest . . . 28

3.1.6 GuiGameControllerTest . . . 29

3.1.7 Vista e audio . . . 30

3.2 Metodologia di lavoro . . . 31

3.2.1 Workflow . . . 31

3.2.2 Version Control . . . 31

3.2.3 Rossi . . . 31

3.2.4 Tontini . . . 32

3.3 Note di sviluppo . . . 33

3.3.1 Rossi . . . 33

3.3.2 Tontini 38

4 Sorgenti **41**

4.1 Link Github 41

4.2 Compilazione e avvio 41

4.3 Crediti 42

Capitolo 1

Analisi

Il team si propone di sviluppare un gioco di carte a turni ispirato alle regole ufficiali dell'UNO, in cui un giocatore umano sfida da uno a nove avversari controllati dal computer.

Il titolo dell'applicativo, **Ultima Carta**, richiama il momento decisivo di ogni mano: quando a un giocatore resta una sola carta deve dichiararlo, premendo il pulsante *Ultima!*, prima di chiudere la mano. Il nome è anche quello del package radice del codice, `lastcard`.

Ultima Carta è un gioco in cui ogni giocatore, a turno, deve liberarsi delle proprie carte giocandone una dello stesso colore, numero o simbolo di quella in cima alla pila degli scarti, oppure un jolly; chi non può giocare pesca. Le carte speciali modificano il giro: fanno saltare il turno, invertono il senso di gioco, costringono a pescare o permettono di scegliere il colore.

Il gioco si fonda su:

- un modello di gioco unico, indipendente dall'interfaccia, che applica tutte le regole e rifiuta le mosse non valide;
- due modalità: la **Classica**, con il mazzo da 108 carte, e la **No Mercy**, con un mazzo da 168 carte, penalità che si sommano ed eliminazione dei giocatori;
- avversari CPU che decidono le mosse guardando soltanto ciò che vedrebbe un giocatore umano;
- una partita a punti, giocata su una o più mani fino a un obiettivo;
- due interfacce sullo stesso modello: una finestra grafica in stile cabinato arcade e una versione da terminale.

Il flusso di gioco prevede che l'utente, dal menu principale, apra la schermata *Nuova partita* e scelga modalità, numero di avversari e obiettivo. Al tavolo gioca le proprie carte con un clic, pesca dal mazzo e dichiara *Ultima!*, mentre le CPU muovono una alla volta, distanziate da una breve attesa, così che si possa seguire ciò che accade.

La partita termina quando un giocatore resta senza carte oppure, nella partita a punti, quando qualcuno raggiunge il punteggio stabilito.

Infine il gioco integra una schermata con le regole complete, le opzioni di schermo e audio (schermo intero, muto, volume), effetti sonori e musica di sottofondo.

1.1 Requisiti

1.1.1 Requisiti funzionali

- Il gioco consente di affrontare da 1 a 9 avversari controllati dal computer, per un totale di 2–10 giocatori al tavolo.
- Modalità **Classica**: mazzo da 108 carte e regole ufficiali (Salta turno, Inverti, Pesca 2, Jolly, Jolly Pesca 4); con due soli giocatori la carta Inverti vale come Salta turno.
- Modalità **No Mercy**: mazzo da 168 carte, accumulo delle penalità, pesca obbligata fino a una carta giocabile, scambio della mano giocando un 7, rotazione delle mani giocando uno 0, eliminazione di chi raggiunge 25 carte e otto nuove carte speciali, tra cui il Jolly Rimescola Mani ideato dal gruppo.
- Dichiarazione di *Ultima!* prima di giocare la penultima carta: chi se ne dimentica pesca due carte di penalità.
- Pesca con scelta: se la carta pescata è giocabile, il giocatore può giocarla subito oppure tenerla e passare il turno.
- Scelta del colore dopo un jolly e, nella No Mercy, scelta dell'avversario con cui scambiare la mano.
- Partita a punti: mano singola oppure più mani fino a 100 o 500 punti (1000 o 2000 nella No Mercy). Chi chiude la mano incassa il valore delle carte rimaste agli avversari.
- Avversari CPU che giocano una mossa alla volta, con un'attesa di due secondi fra una mossa e l'altra.
- Evidenziazione delle carte giocabili, popup per le carte speciali, riga che racconta l'ultimo evento e percorso del giro con le frecce del senso di gioco.
- Schermata delle regole delle due modalità.
- Opzioni di schermo intero, muto e volume, disponibili anche durante la partita, che nel frattempo resta in pausa; possibilità di abbandonare la partita in corso.
- Effetti sonori per le azioni principali e musica di sottofondo nel menu.
- Versione da terminale con gli stessi menu e le stesse regole della versione grafica.

1.1.2 Requisiti non funzionali

Oltre alle funzionalità principali, l'applicazione dovrà garantire:

- un modello di gioco indipendente dall'interfaccia, riutilizzabile da viste diverse senza modifiche;
- correttezza delle regole: ogni mossa non valida viene rifiutata e lascia lo stato della partita invariato;
- partite riproducibili a parità di seme casuale, per i test e per la ricerca degli errori;
- un'interfaccia chiara e leggibile, che si adatta dalla finestra da 960×640 allo schermo intero.
- facilità di estensione con nuove carte o nuove regole;
- funzionamento in locale, senza connessione a Internet e senza librerie esterne oltre al JDK.

1.1.3 Diagramma dei Casi d'Uso

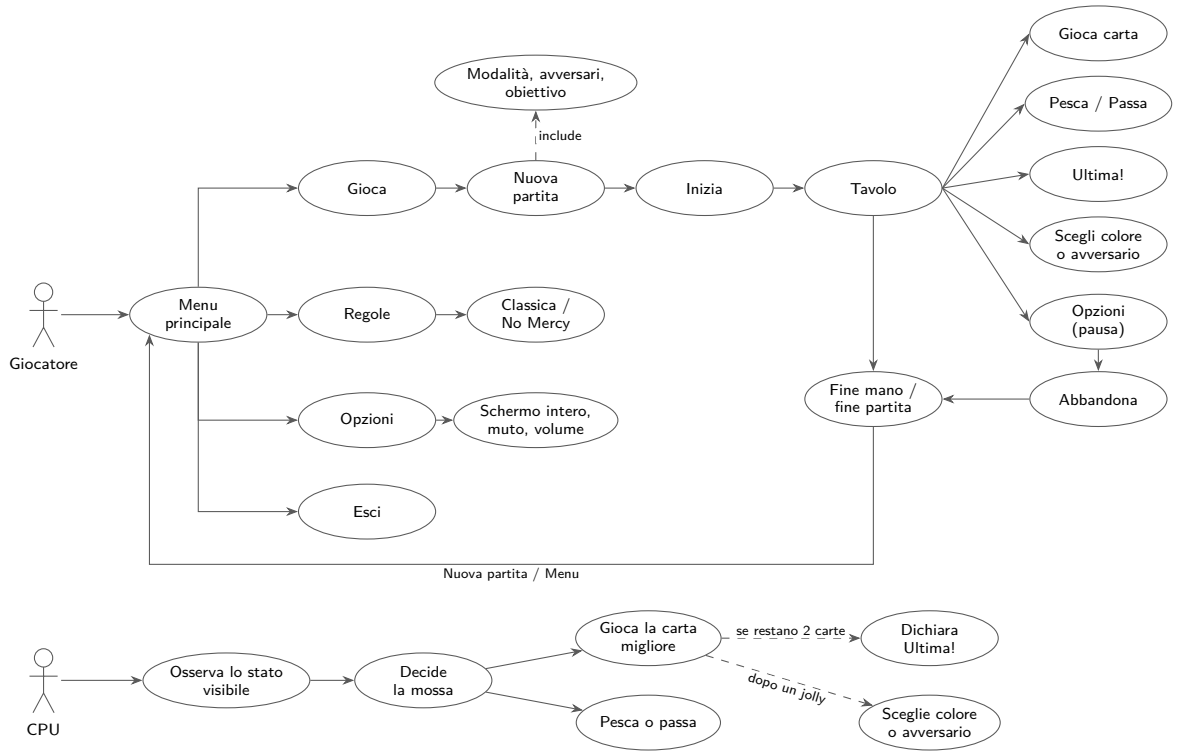


Figura 1.1: Diagramma dei Casi d'Uso

- **Attori coinvolti**
  - **Giocatore** – L'utente che naviga nei menu, configura la partita e gioca al tavolo.
  - **CPU** – Il giocatore automatico che, quando è il suo turno, decide la mossa a partire dallo stato visibile della partita.
- **Casi d'uso principali**
  - **Per il Giocatore**
    - Avvia nuova partita – Dal menu principale seleziona *Gioca*, sceglie modalità, numero di avversari e obiettivo nella schermata *Nuova partita* e preme *Inizia*.
    - Consulta le regole – Dal menu principale apre *Regole* e passa dalla modalità Classica alla No Mercy.
    - Cambia le opzioni – Dal menu o dal tavolo attiva lo schermo intero, il muto e regola il volume.
    - Gioca il turno – Al tavolo gioca una carta, pesca dal mazzo, passa dopo aver pescato o dichiara *Ultima!*.
    - Risponde a una scelta – Dopo un jolly sceglie il colore; nella No Mercy sceglie l'avversario dello scambio o il colore della Roulette.
    - Prosegue la partita a punti – A fine mano consulta la classifica e passa alla mano successiva.
    - Abbandona la partita – Dalle opzioni del tavolo torna al menu principale.

  - **Per la CPU**
    * Osserva lo stato visibile – Legge la propria mano, la carta in cima, il colore attivo e il numero di carte degli avversari.
    * Decide la mossa – Gioca la carta migliore, oppure pesca; dichiara *Ultima!* quando serve.
    * Compie le scelte – Dichiara il colore dopo un jolly e sceglie l'avversario dello scambio.

- **Relazioni tra attori e casi d'uso**
  - Il **Giocatore** interagisce con l'intero sistema dei menu e prende le decisioni di gioco al proprio turno.
  - La **CPU** agisce in autonomia, secondo una strategia predefinita, usando le stesse mosse che sono disponibili al giocatore umano.

1.2 Analisi e Modello del dominio

Il sistema è progettato per gestire una partita di carte a turni fra più giocatori, in cui le regole cambiano con la modalità scelta ma il flusso del turno resta lo stesso.

L'architettura mira a favorire la separazione fra le regole e la loro rappresentazione, così che la stessa partita possa essere giocata in una finestra grafica, da terminale o, nei test, senza alcuna interfaccia.

L'entità centrale del sistema è rappresentata dalla classe **Game**, responsabile dello svolgimento della partita: distribuisce le carte, tiene il turno e il senso di gioco, applica gli effetti delle carte e decide quando la mano è chiusa. Essa riceve le azioni dei giocatori sotto forma di mosse e ne comunica l'esito con eventi.

Il dominio si compone dei seguenti elementi principali:

- **Carte (`Card`):** ogni carta ha un colore, un simbolo e un valore in punti, sa dire se può essere giocata sopra la carta in cima e applica da sé il proprio effetto alla partita. Si dividono in carte numero, carte azione colorate e jolly neri.
- **Mazzo e pila degli scarti (`Deck`, `DiscardPile`):** il mazzo si pesca dalla cima e, una volta esaurito, si rigenera con gli scarti tranne l'ultima carta giocata.
- **Giocatori (`Player`):** hanno un nome, una mano, la dichiarazione di *Ultima!*, il punteggio della partita a punti e, nella No Mercy, lo stato di eliminazione.
- **Regole (`GameRules`):** descrivono come dati ciò che distingue una modalità dall'altra: accumulo delle penalità, soglia di eliminazione, scambio 7, rotazione 0, pesca obbligata, punti per vincere e tipo di mazzo.
- **Mosse (`Move`):** le azioni possibili di un giocatore (giocare una carta, pescare, passare, dichiarare *Ultima!*, scegliere un colore, scambiare la mano), uguali per umani e CPU.
- **Eventi e stato visibile (`GameEvent`, `VisibleState`):** ciò che la partita rende pubblico. Gli eventi raccontano cosa è successo senza mai rivelare carte nascoste; lo stato visibile mostra la mano di chi guarda e, degli avversari, solo il numero di carte.
- **Giocatori CPU (`CpuPlayer`):** trasformano lo stato visibile in una lista di mosse, secondo una strategia intercambiabile.

La gestione del turno avviene interamente in **Game**, che valida ogni mossa e la esegue oppure la rifiuta con un'eccezione, lasciando lo stato invariato.

In sintesi, il modello del dominio è composto dalle entità principali `Game`, `Card`, `Deck`, `Player`, `GameRules`, `Move` e `GameEvent`. Ciascuna entità è responsabile di una specifica parte del comportamento del gioco, contribuendo a definire un'architettura modulare, facilmente manutenibile ed estendibile.

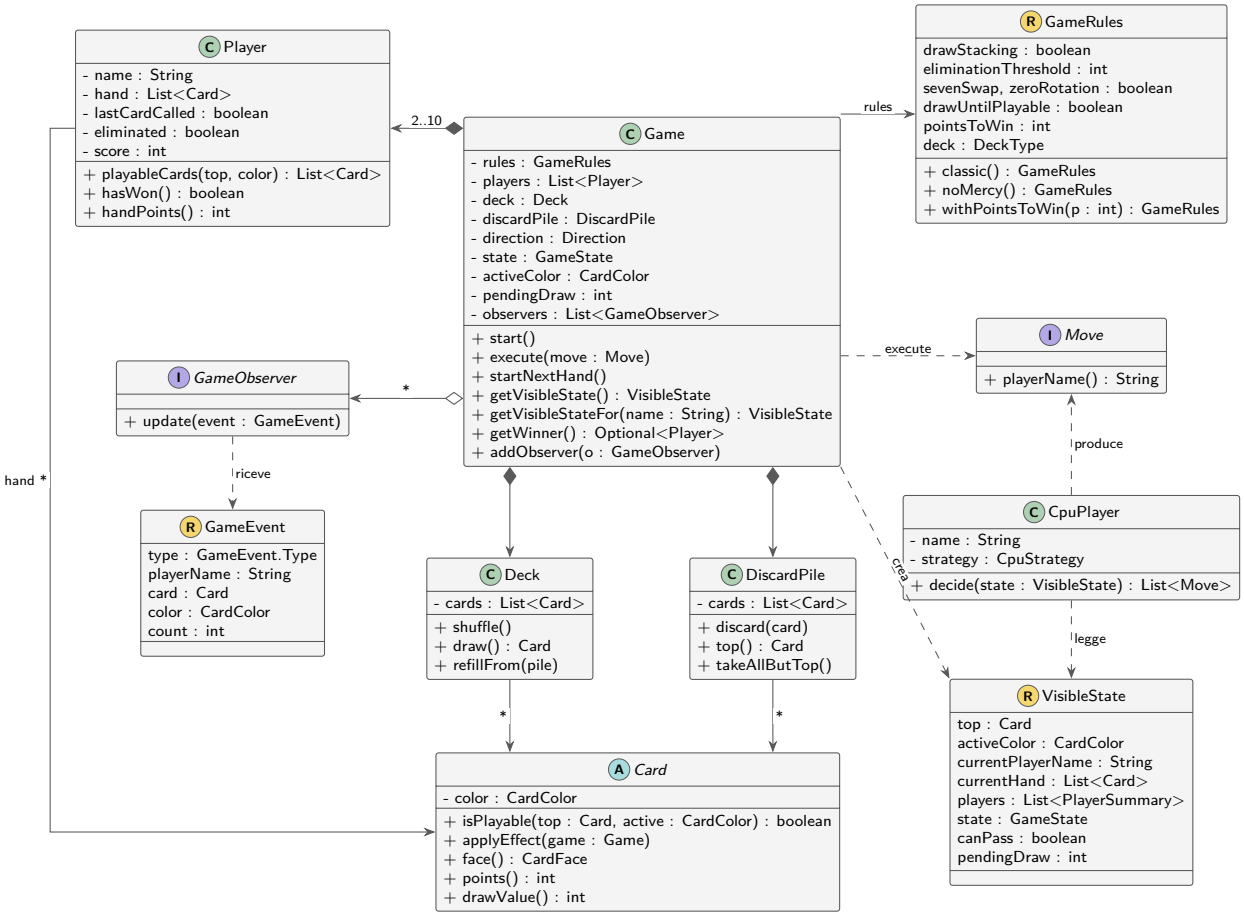


Figura 1.2: Schema UML del dominio

Capitolo 2

Design

2.1 Architettura

Per lo sviluppo è stato adottato il pattern Model-View-Controller.

Il **Model** gestisce lo stato della partita e le sue regole, offrendo metodi per eseguire le mosse e per leggere ciò che è visibile, ad esempio: **carte**, **mazzo**, **giocatori**, **regole** ed **eventi**.

La **View** si occupa invece della rappresentazione delle informazioni contenute nel Model, presentandole all'utente tramite i **menu** e il **tavolo** della finestra grafica oppure tramite il **terminale**.

L'interazione dell'utente avviene tramite la View, ma la sua elaborazione è affidata al **Controller**, che traduce ogni gesto in una mossa del Model e, a mossa eseguita, chiede alla View di mostrare il nuovo stato.

La dipendenza fra i tre livelli va in un solo verso: il package `model` non importa nulla da `view` né da `controller`. I cambiamenti della partita arrivano alle viste come eventi, attraverso l'interfaccia `GameObserver`, e le viste ricevono soltanto uno stato in sola lettura (`VisibleState`). Il Controller, a sua volta, conosce le viste solo attraverso interfacce (`GameView`, `TableView`, `MenuView`), così da poter essere provato con viste finte e senza Swing.

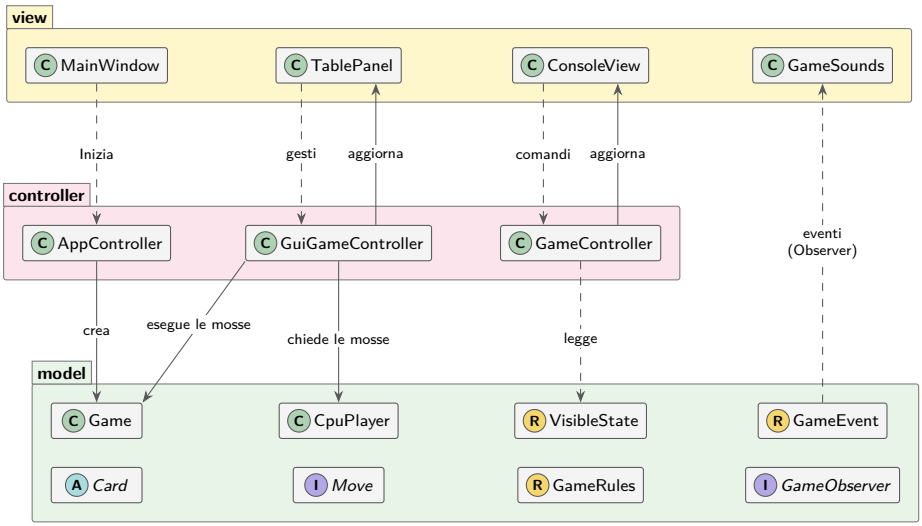


Figura 2.1: MVC design pattern

2.1.1 Model

Il Model si occupa di gestire le modalità di accesso e modifica dei dati della partita, determinando coerentemente come le mosse dei giocatori influenzino lo stato corrente.

Scopo del Model è quindi fornire al Controller un accesso controllato alla partita: un unico punto di ingresso per le azioni, il metodo `Game.execute(Move)`, e due modi per osservarla, gli eventi e lo stato visibile.

Per fornire tale funzionalità è stato scelto di utilizzare un'entità `Game`, responsabile di mantenere lo stato corrente, che include i principali componenti del sistema: le **carte** (numeri, carte azione e jolly delle due modalità), il **mazzo** e la **pila degli scarti**, i **giocatori** con le loro mani e i loro punteggi, le **regole** della modalità scelta e lo **stato del turno** (giocatore corrente, senso di gioco, colore attivo, penalità in corso).

Nel Model si trovano anche i giocatori CPU (`CpuPlayer`), che non modificano la partita: a partire dallo stato visibile restituiscono le mosse da eseguire.

2.1.2 View

La View consiste nelle schermate dell'applicazione e si occupa della parte grafica, della User Experience e dell'interazione con l'utente.

È compito delle varie View raccogliere i gesti dell'utente (un clic su una carta, sul mazzo o su un pulsante, un comando digitato) e inoltrarli al rispettivo Controller, in attesa del nuovo stato da mostrare.

Si è scelto di progettare View e Controller in modo indipendente dal framework grafico utilizzato: il Controller parla con interfacce del package `view` e non con classi Swing. La prova pratica di questa scelta è la vista da terminale, che gioca la stessa partita con lo stesso modello; un cambio di libreria grafica richiederebbe di riscrivere soltanto la View.

La struttura del package `view` è suddivisa in tre parti: `view.gui` per la finestra Swing (menu, *Nuova partita*, *Regole*, *Opzioni* e il tavolo, con le carte disegnate a codice), `view.console` per il terminale e `view.audio` per musica ed effetti sonori. Anche l'audio è un osservatore della partita, come le viste: il modello non sa che esiste.

2.1.3 Controller

Il Controller è quella componente cui spetta gestire in maniera consequenziale le interazioni da parte dell'utente nella View, comunicando quindi al Model il cambiamento richiesto.

Una volta che il Model avrà completato l'elaborazione della mossa, il Controller avviserà la propria View, in modo tale che quest'ultima possa aggiornarsi in maniera coerente con le regole specificate dal Model.

Si è scelto di evitare un singolo Controller, perché la finestra grafica e il terminale hanno bisogno di un flusso diverso e perché un unico controller avrebbe avuto un eccessivo grado di responsabilità. Nel package `controller` si trovano infatti tre controller:

- **AppController**: il controller del menu; dalle scelte di *Nuova partita* crea regole, giocatori e partita e la affida al controller del tavolo.
- **GuiGameController**: il controller della partita nella finestra; reagisce ai gesti del giocatore e fa muovere le CPU una mossa alla volta, distanziate da un'attesa.
- **GameController**: il controller della partita da terminale, che alterna le mosse delle CPU e la lettura dei comandi digitati.

Completano il package le interfacce che il Controller espone alle viste (`MenuActions`, `TableActions`), l'astrazione del temporizzatore (`TurnScheduler`) e le impostazioni scelte dall'utente (`GameSettings`, `GameMode`, `GameGoal`).

2.2 Design dettagliato

2.2.1 Rossi

Nella divisione dei compiti concordata all'inizio mi è stata assegnata l'area "cosa succede": il motore di gioco, con i turni, gli stati, le mosse e gli eventi, e le decisioni delle CPU. Con l'avanzare del progetto mi sono occupato anche della vista grafica: il disegno delle carte, il tavolo di gioco, lo stile cabinato, le opzioni e l'audio, insieme al controller che fa giocare la partita nella finestra.

In particolare ho implementato le classi `Game` e `Player`, le mosse, il sistema di eventi, le regole No Mercy e la partita a punti all'interno del motore, la strategia delle CPU, `GuiGameController` e quasi tutto il package `view.gui` e il package `view.audio`.

Game

La classe `Game` rappresenta il nucleo della logica di gioco e funge da *facciata* verso il resto dell'applicazione: dall'esterno si vedono soltanto `start()`, `execute(Move)`, `startNextHand()` e i metodi di lettura dello stato.

Tra le funzionalità principali troviamo:

- **Esecuzione delle mosse:** `execute(Move)` smista ogni mossa al metodo che la gestisce con uno `switch` sui tipi della `sealed interface Move`; il compilatore verifica che tutti i casi siano coperti.
- **Validazione:** ogni mossa viene controllata prima di modificare qualcosa (turno, indice della carta, giocabilità, penalità in corso, stato atteso). Una mossa non valida solleva `IllegalArgumentException` o `IllegalStateException` e lascia la partita com'era.
- **Gestione del turno:** indice del giocatore corrente e senso di gioco (`Direction`); il metodo `stepFrom` calcola il giocatore successivo saltando gli eliminati. Con due giocatori la carta Inverti vale come Salta turno, come da regolamento.
- **Prima carta:** la carta scoperta all'inizio vale come giocata dal mazziere, quindi i suoi effetti colpiscono il primo giocatore. Se è un Jolly il colore lo sceglie il primo giocatore; se è un Jolly Pesca 4 il primo giocatore pesca e il colore lo sceglie il mazziere.
- **Pesca e Ultima!:** chi pesca una carta giocabile può giocarla subito o tenerla con la mossa `Pass`; chi resta con una carta senza aver dichiarato *Ultima!* pesca due carte di penalità.
- **Chiusura della mano:** `finishHand` decide se la partita è finita oppure, nella partita a punti, assegna i punti a chi ha chiuso la mano e attende la mano successiva.

Lo stato della partita è rappresentato dall'enum `GameState`. Oltre a `IN_PROGRESS` esistono tre stati di attesa, in cui la partita aspetta una scelta da un giocatore preciso: il colore dopo un jolly, l'avversario dello scambio 7 e il colore della Roulette. Senza uno stato esplicito questa situazione sarebbe diventata un flag booleano sparso nel codice. In questi stati il giocatore che deve agire non è quello di turno: dopo un Jolly Pesca 4, ad esempio, il turno è già passato a chi ha pescato, ma il colore lo sceglie chi ha giocato la carta. Il metodo privato `getActingPlayer()` risolve questa differenza.

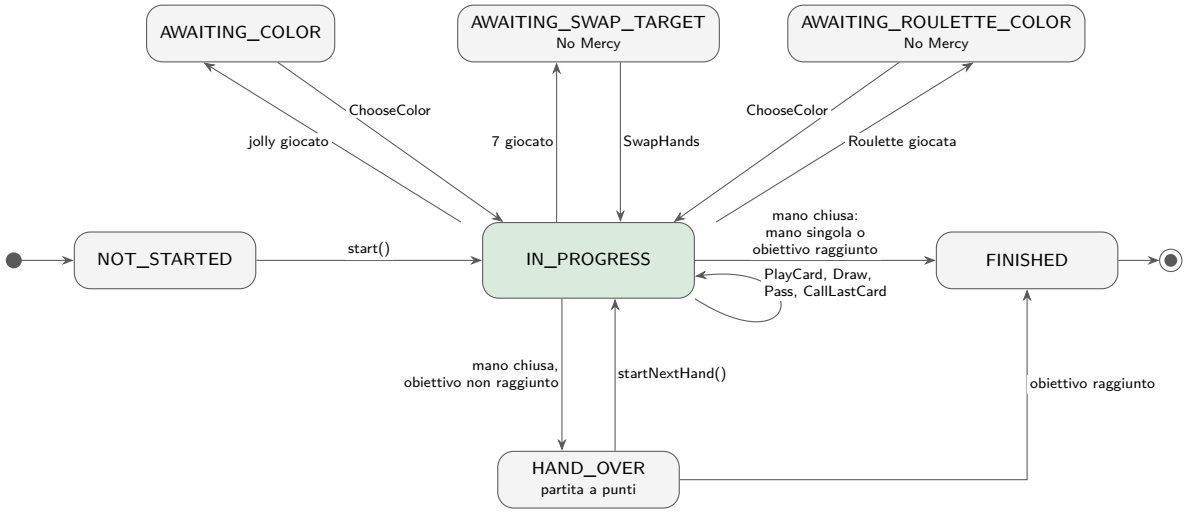


Figura 2.2: Gli stati della partita (`GameState`) e le mosse che li fanno cambiare

Mosse

Ogni azione di un giocatore è un oggetto che implementa `Move`, un'interfaccia `sealed` che ammette soltanto sei implementazioni, tutte record immutabili:

- **PlayCard**: gioca la carta in una posizione della mano;
- **Draw**: pesca (con una penalità in corso, prende tutto il totale);
- **Pass**: tiene la carta appena pescata e passa il turno;
- **CallLastCard**: dichiara *Ultima!*;
- **ChooseColor**: dichiara il colore dopo un jolly o per la Roulette;
- **SwapHands**: nello scambio del 7, indica l'avversario con cui scambiare la mano.

È un'applicazione del pattern *Command*: l'intenzione del giocatore viene rappresentata come dato e consegnata a un unico punto di esecuzione, `Game.execute(Move)`. Umani e CPU usano esattamente le stesse mosse, quindi la partita non distingue chi le ha prodotte e applica a tutti le stesse verifiche. I costruttori compatti dei record controllano già i vincoli che non dipendono dalla partita: un indice negativo, il colore nero in `ChooseColor`, uno scambio con sé stessi in `SwapHands`. Rendere `Move sealed` permette a `Game` di smistare le mosse senza ramo `default`: se si aggiungesse una mossa, il compilatore segnalerebbe ogni punto da aggiornare.

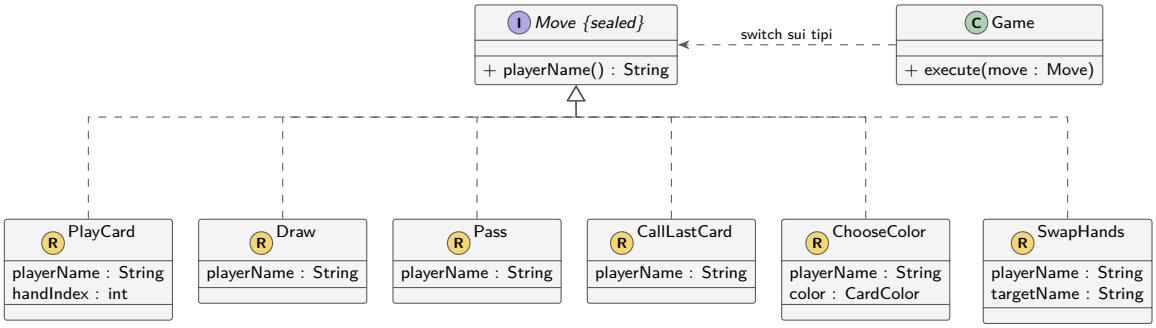


Figura 2.3: UML delle mosse

Eventi e stato visibile

La partita comunica con l'esterno attraverso il pattern *Observer*. L'interfaccia `GameObserver` ha un solo metodo, `update(GameEvent)`, e può quindi essere implementata anche con una lambda (i test, ad esempio, registrano `events::add`); `Game` mantiene la lista degli osservatori e li notifica dopo ogni cambiamento, iterando su una copia della lista così che un osservatore possa rimuoversi durante la notifica.

`GameEvent` è un record con il tipo dell'evento (un enum di diciotto valori, da `GAME_STARTED` a `HAND_WON`) e i dati che lo descrivono; i campi che un tipo non usa valgono `null`. Gli eventi si creano con metodi statici con nome (`cardPlayed`, `cardsDrawn`, `penaltyStacked`...), visibili solo all'interno del package `model`. Gli eventi sono pubblici per tutti i giocatori, quindi dicono quante carte sono state pescate ma mai quali; il testo da mostrare lo decide la vista, in italiano.

Lo stato che una vista o una CPU possono leggere è il record `VisibleState`, definito da Tontini: la carta in cima, il colore attivo, chi deve agire, la mano di chi guarda, un `PlayerSummary` per ogni giocatore e lo stato della partita. Io vi ho aggiunto la possibilità di passare, la penalità in corso e, per il tavolo grafico, il metodo `getVisibleStateFor(nome)`, che mostra sempre la mano del giocatore umano anche quando sta agendo una CPU.

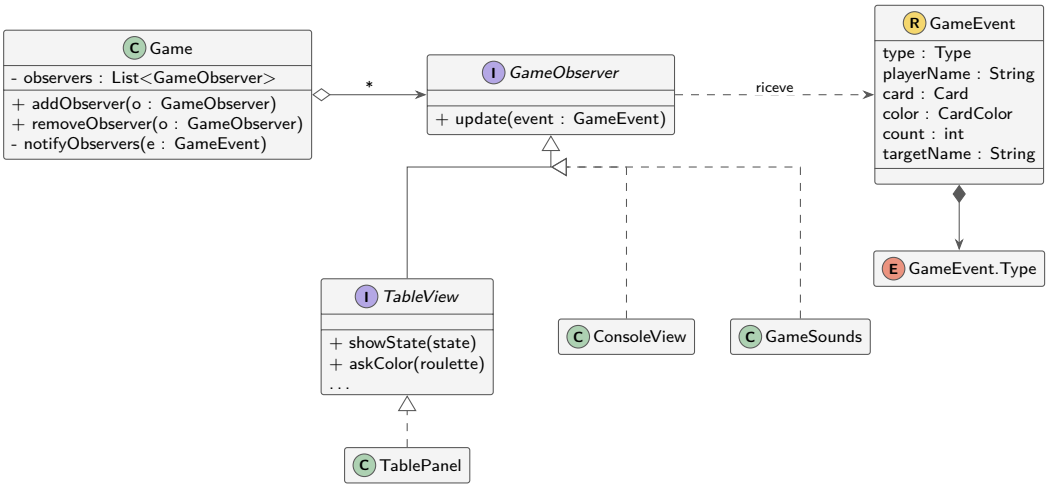


Figura 2.4: UML del sistema di eventi (Observer)

Regole No Mercy nel motore

Le regole No Mercy vivono in `Game` e si attivano in base ai campi di `GameRules`:

- **Accumulo:** le carte Pesca chiamano sempre `penalizeNext(n)`; è `Game` a decidere se far pescare subito o sommare la penalità, che il giocatore successivo deve rilanciare o pescare per intero.
- **Eliminazione:** chi arriva a 25 carte esce dal giro e la sua mano va sotto la cima degli scarti (`DiscardPile.addUnderTop`); se resta un solo giocatore, la mano è sua.
- **Scambio 7 e rotazione 0:** dipendono dalle regole e non dal tipo di carta, quindi sono riconosciuti in `Game.resolveCard` dal simbolo della carta, dopo il suo effetto.
- **Salta tutti, Scarta tutto e Roulette:** operazioni dedicate di `Game` (`skipAll`, `discardAllOfColor`, `startColorRoulette`) richiamate dall'effetto delle carte.

La regola che stabilisce se una carta si può giocare, tenendo conto della penalità accumulata, è isolata nella classe di utilità `PlayRules`. La usano sia `Game`, per validare le mosse, sia `CpuPlayer`, per sceglierle, sia il tavolo grafico, per evidenziare le carte giocabili: tenerla in un solo punto evita che la CPU proponga mosse che la partita poi rifiuta.

CPU

La classe `CpuPlayer` decide le mosse di un giocatore automatico guardando soltanto il `VisibleState`, cioè ciò che vedrebbe un giocatore umano: non può barare e funzionerebbe allo stesso modo con qualunque motore capace di produrre quello stato. Per questo non estende `Player` e non modifica nulla: il metodo `decide(VisibleState)` restituisce la lista delle mosse da eseguire, di solito una sola, oppure due quando gioca la penultima carta (prima `CallLastCard`, poi `PlayCard`).

La scelta vera e propria è delegata a un oggetto `CpuStrategy`, secondo il pattern *Strategy*. L'interfaccia definisce tre decisioni: quale carta giocare fra quelle giocabili (oppure nessuna, per pescare), quale colore dichiarare e con quale avversario scambiare la mano. L'implementazione fornita, `BasicCpuStrategy`, gioca sempre se può, tiene i jolly per ultimi e fra le carte colorate si libera prima di quelle che valgono più punti; dopo un jolly dichiara il colore che ha in maggior numero e nello scambio del 7 prende la mano dell'avversario con meno carte. Una strategia più astuta si potrebbe aggiungere senza modificare `CpuPlayer`.

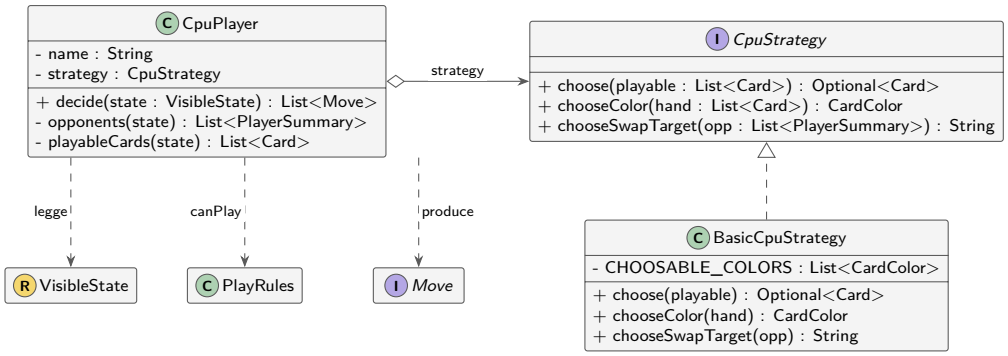


Figura 2.5: UML dei giocatori CPU (Strategy)

Controller del tavolo

`GuiGameController` è il controller della partita nella finestra e implementa `TableActions`, l'insieme dei gesti che il tavolo può inoltrare: giocare una carta, pescare, passare, dichiarare *Ultima!*, scegliere colore o avversario, passare alla mano successiva, mettere in pausa, riprendere e lasciare il tavolo. A differenza del controller per il terminale non attende mai l'input in un ciclo: reagisce agli eventi e, dopo ogni mossa di chiunque, esegue la stessa procedura (`afterMove`): mostra lo stato visto dal giocatore umano, poi chiede una scelta all'umano, oppure programma la mossa della CPU di turno, oppure mostra il vincitore.

Le mosse delle CPU sono distanziate da un'attesa di due secondi tramite l'interfaccia `TurnScheduler`, che ha due sole operazioni: `schedule(ritardo, compito)` e `cancel()`. Nella finestra è implementata da `SwingTurnScheduler` con un `javax.swing.Timer` che scatta una volta sola, così il compito gira sul thread grafico e la partita è toccata sempre da un solo thread; nei test è sostituita da un temporizzatore finto che esegue la mossa solo quando lo chiede il test. La pausa del form *Opzioni* annulla la mossa in attesa e la ripresa ricomincia da `afterMove` con un'attesa intera. Una mossa rifiutata dal modello viene trasformata in un messaggio in italiano: con una penalità in corso, ad esempio, il messaggio spiega con quali carte si può rilanciare o quante carte si devono pescare.

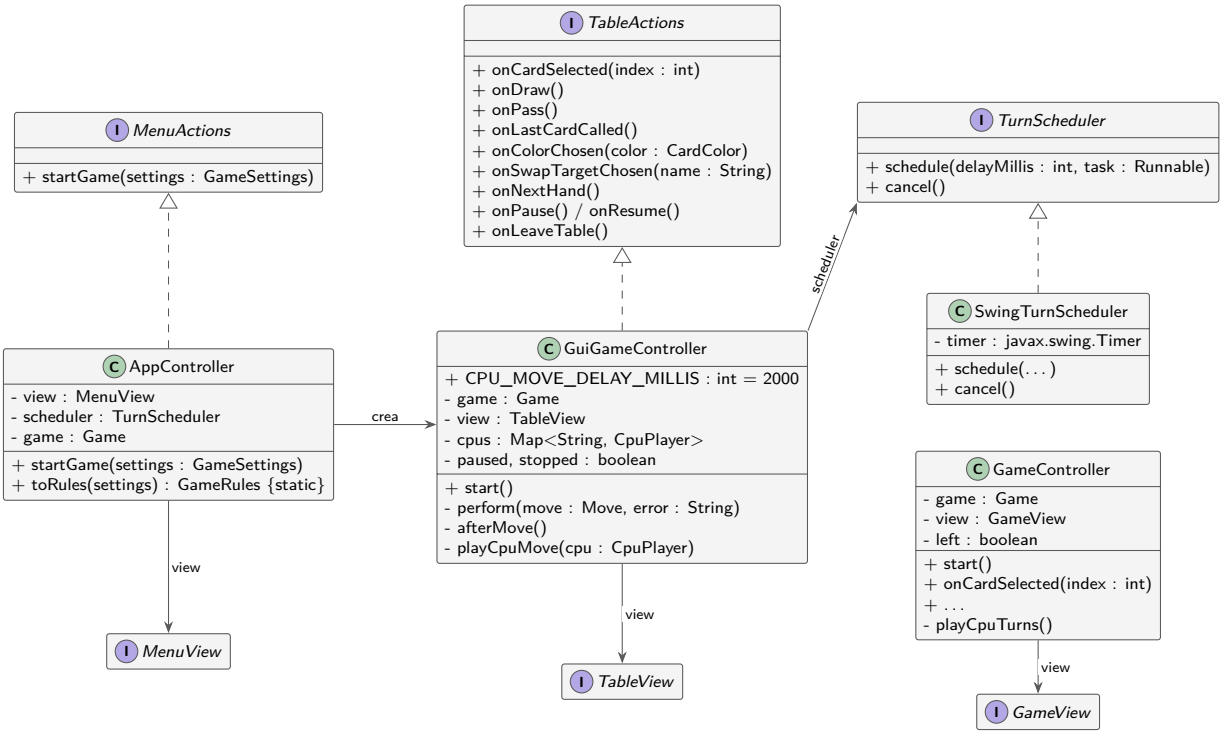


Figura 2.6: UML del Controller (`AppController` e `GameController` sono descritti nella sezione di Tontini)

Carte disegnate a codice

Il gruppo ha deciso di non usare un'immagine per ogni carta ma di disegnarle a codice con Java 2D. Ho introdotto per questo il record `CardFace`, con colore, simbolo (l'enum `FaceSymbol`) ed etichetta degli angoli, e il metodo astratto `face()` in `Card`. È il contratto fra modello e vista: ogni carta sa descriversi e la vista sa disegnare una descrizione, senza interrogare il tipo concreto della carta con `instanceof`.

Le carte le disegna `CardRenderer` su un `Graphics2D`, nello stile di un vecchio monitor (cornice scura, righe di scansione, cifre e simboli a pixel). Nessuna misura è fissa: tutto deriva dalla larghezza richiesta, così la stessa classe serve la mano del giocatore, la pila degli scarti e le miniature. Il disegno di ogni simbolo è registrato in una `EnumMap<FaceSymbol, Glyph>`, dove `Glyph` è un'interfaccia funzionale privata e i valori sono riferimenti a metodi: aggiungere una carta significa registrare una voce, non modificare uno `switch`. Se per un simbolo manca il disegno, si ottiene un'eccezione con il nome del simbolo invece di una carta disegnata a metà. I simboli a blocchi sono matrici di pixel in `PixelGlyphs` e i colori vivono tutti in `ArcadePalette`.

Poiché ridisegnare ogni carta a ogni `repaint` sarebbe costoso, `CardImageCache` conserva le immagini già disegnate in una mappa la cui chiave è un record con la `CardFace`, la larghezza in pixel e l'evidenziazione: le 108 carte del mazzo classico hanno solo 54 facce diverse. La larghezza in pixel tiene conto del ridimensionamento dello schermo del sistema operativo (125%, 150%), così le carte restano nitide su tutti gli schermi. La classe `CardPreview` produce un'immagine con tutte le facce, usata durante lo sviluppo per controllare il disegno.

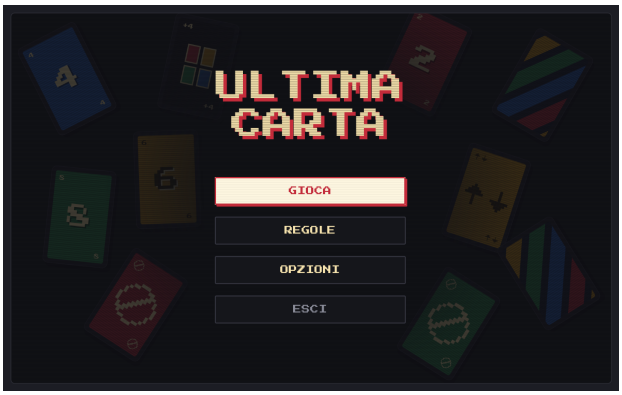


Figura 2.7: Le facce delle carte generate da `CardPreview`: il mazzo classico, le carte giocabili evidenziate, il dorso e le carte della No Mercy

Schermate e stile cabinato

Sulla finestra e sulla navigazione impostate da Tontini ho costruito lo stile cabinato arcade di tutte le schermate. Ogni schermata estende `ArcadeBackgroundPanel`, che disegna la cornice scura del cabinato e, al suo interno, lo sfondo ridimensionato senza deformazioni; i contenuti stanno in un riquadro scuro (`ArcadeBox`) e i titoli sono scritti a pixel da `ArcadeTitle`. Le misure di testi e controlli sono moltiplicate per un fattore di scala calcolato da `ScreenScale` rispetto alla finestra di base (960×640), così le proporzioni restano le stesse fino allo schermo intero.

L'aspetto dei componenti Swing è stato personalizzato con *UI delegate* dedicati, senza sottoclassi dei componenti: `ArcadeButtonUI` (pulsanti piatti con righe di scansione e quattro varianti, fra cui `ALERT` per *Ultima!*), `ArcadeCheckBoxUI` e `ArcadeSliderUI`. Il carattere a pixel *Press Start 2P* viaggia dentro al jar ed è caricato una sola volta da `ArcadeFont`; `FocusIndicator` mostra la freccetta del focus solo a chi usa la tastiera.



(a) Menu principale



(b) Nuova partita

Figura 2.8: Due schermate della finestra grafica

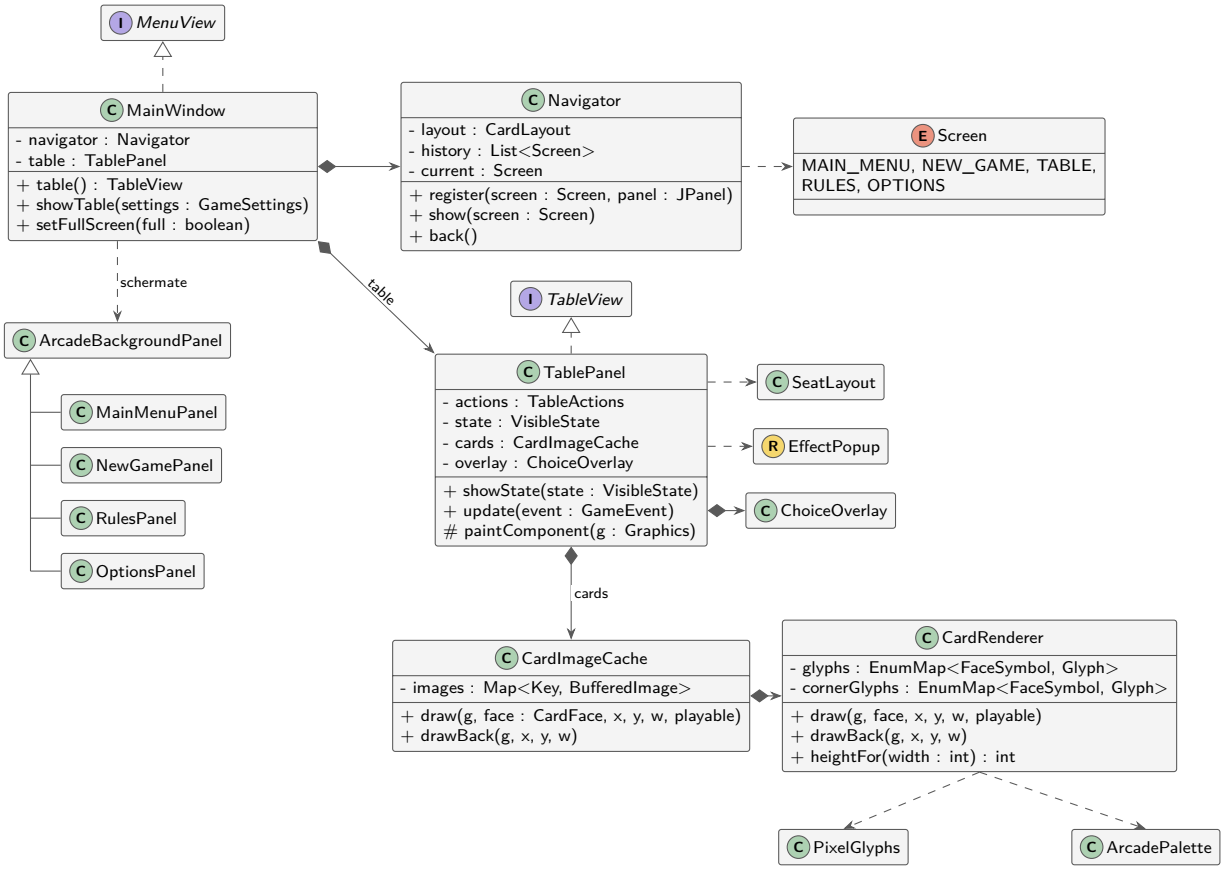


Figura 2.9: UML della vista grafica

Tavolo di gioco

La classe `TablePanel` è il tavolo e implementa `TableView`. Mostra soltanto ciò che riceve dal controller, cioè lo stato visto dal giocatore umano e gli eventi della partita, e non conosce `Game`: ogni gesto (una carta, il mazzo, *Passa*, *Ultima!*, una scelta nel form) diventa una chiamata a `TableActions`.

Il tavolo è disegnato interamente nel metodo `paintComponent`: gli avversari su un arco di ellisse, collegati da un percorso chiuso (una spline di Catmull-Rom) con le frecce del senso di gioco; al centro il mazzo e gli scarti, con il colore attivo e la penalità in corso; in basso la mano a ventaglio, con le carte giocabili bordate di crema. La posizione degli avversari è calcolata da `SeatLayout`, una classe di sola geometria. Per riconoscere la carta cliccata, ogni carta della mano viene memorizzata con la trasformazione affine con cui è stata ruotata (record `HandCard`); il punto del clic viene riportato nel sistema della carta con la trasformazione inversa.

Il senso di gioco non fa parte dello stato visibile: il tavolo lo ricava dagli eventi `DIRECTION_REVERSED`, che riceve come osservatore. Gli eventi di una mossa vengono raccolti e, all'arrivo del nuovo stato, trasformati da `EffectPopup` in un popup di un secondo che fa notare le carte speciali ("+4", "CAMBIO GIRO", "SALTA TUTTI", la penalità che sale…), mentre `TableTexts` produce la frase che racconta l'ultimo evento. Le scelte (colore, avversario dello scambio, fine mano, fine partita, opzioni) si aprono in un `ChoiceOverlay`, un velo scuro che copre il tavolo e ne intercetta i clic; nella partita a punti la classifica di fine mano è costruita da `ScoreBoard`.



Figura 2.10: Il tavolo in una partita No Mercy a punti contro quattro CPU, con una penalità in corso

Opzioni e audio

Le impostazioni di schermo intero, muto e volume sono gestite da `Settings`, che le salva con `java.util.prefs.Preferences`; i controlli (`OptionsControls`) sono gli stessi nella schermata *Opzioni* e nel form *Opzioni* del tavolo, che mette in pausa la partita.

`AudioPlayer` riproduce effetti e musica dai file WAV delle risorse con `javax.sound.sampled.Clip`: gli effetti ripartono dall'inizio a ogni richiesta, mentre la musica gira in loop con una dissolvenza in entrata e in uscita calcolata direttamente sui campioni, così il punto di ripetizione non si sente. Se un file manca o il computer non ha l'audio, il lettore lo segnala una sola volta e il gioco prosegue

in silenzio. Il collegamento con la partita è `GameSounds`, un altro `GameObserver` che associa i tipi di evento agli effetti sonori con una `EnumMap`: il modello non sa che esiste l'audio.

2.2.2 Tontini

Nella divisione dei compiti mi è stata assegnata l'area "cosa esiste": i dati e la configurazione del gioco, cioè le carte, il mazzo, la pila degli scarti, la composizione dei mazzi e le regole, insieme al controller e alla vista da terminale. Ho impostato inoltre il progetto Maven, la prima versione della finestra grafica con il menu principale e la navigazione, la schermata *Nuova partita* con il controller del menu e i testi delle regole.

Carte

La classe astratta `Card` rappresenta una carta qualsiasi: ha un colore immutabile e dichiara i metodi che ogni carta deve definire, cioè il simbolo (`symbol()`) e il valore in punti (`points()`). Fornisce inoltre due comportamenti di base che le sottoclassi possono ridefinire: `isPlayable(top, activeColor)`, vero se la carta ha il colore attivo o lo stesso simbolo di quella in cima, e `applyEffect(Game)`, che di base non fa nulla. Due carte sono uguali se hanno la stessa classe, lo stesso colore e lo stesso simbolo: `equals` e `hashCode` sono ridefiniti, così due carte uguali sono intercambiabili nei test e nelle collezioni.

Sotto `Card` si trovano tre famiglie:

- **NumberCard**: le carte da 0 a 9, che valgono il loro numero e non possono essere nere;
- **ActionCard**: le carte azione colorate, da 20 punti: `Skip`, `Reverse`, `DrawTwo` e, per la No Mercy, `ColorDrawFour`, `SkipEveryone` e `DiscardAll`;
- **WildCard**: i jolly neri, da 50 punti, sempre giocabili: `Wild`, `WildDrawFour` e, per la No Mercy, `WildDrawSix`, `WildDrawTen`, `WildReverseDrawFour`, `WildColorRoulette` e `WildShuffleHands`.

L'effetto di ogni carta è un'operazione polimorfica, scritta direttamente nella classe della carta: `applyEffect` riceve la partita e richiama le sue operazioni elementari (`skipNext()`, `reverseDirection()`, `penalizeNext(n)`, `awaitColorChoice()`, `skipAll()`, `discardAllOfColor(c)`, `startColorRoulette()`, `shuffleHands()`). In questo modo `Game` non contiene nessuno `switch` sul tipo di carta e una nuova carta si aggiunge scrivendo una sottoclasse, nel rispetto del principio *Open/Closed*. Il valore di pesca è esposto da `drawValue()`, che vale 0 per le carte senza penalità e serve alle regole di accumulo senza controlli di tipo.

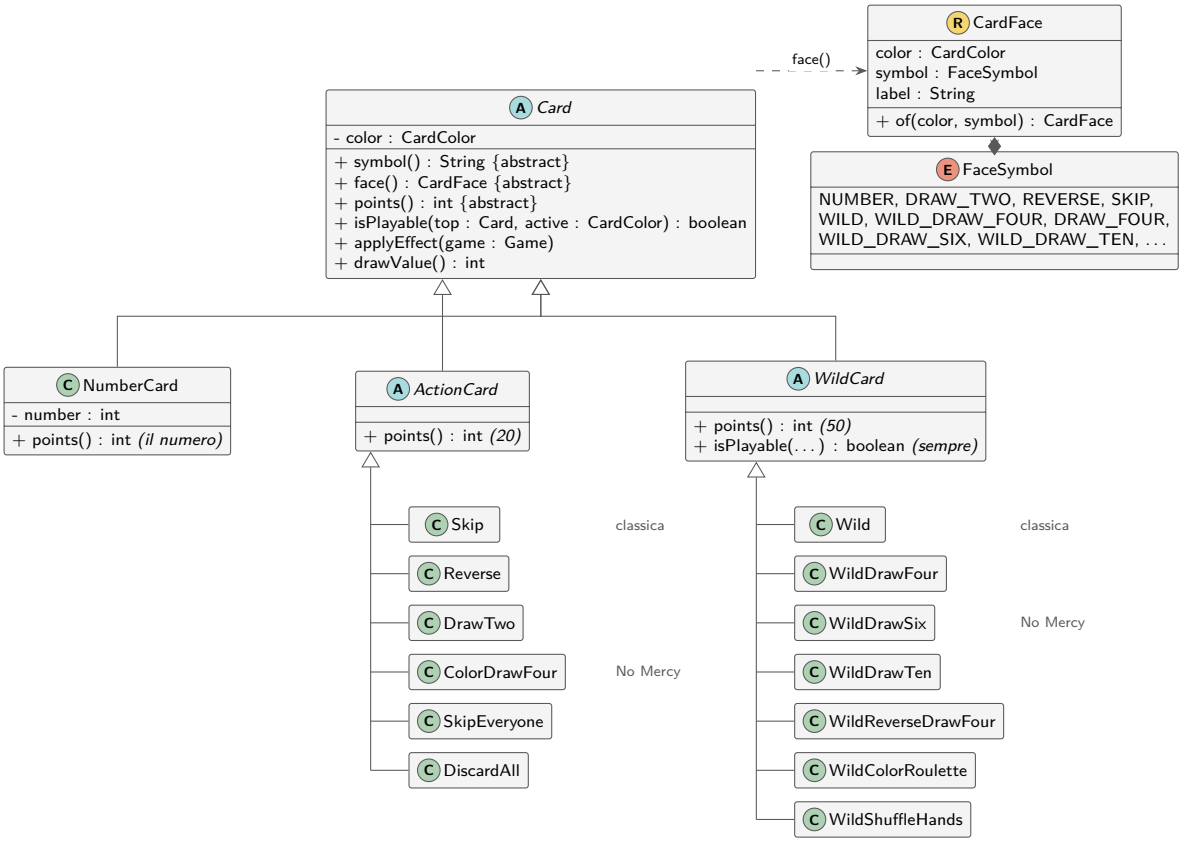


Figura 2.11: UML della gerarchia delle carte

Mazzo, scarti e DeckFactory

La classe `Deck` incapsula la lista delle carte da pescare, con la cima in fondo alla lista, come la struttura vista a lezione. Quando il mazzo finisce, `refillFrom(DiscardPile)` vi rimette gli scarti tranne la carta in cima e rimescola; se anche dopo la rigenerazione mancano carte, si pesca quello che c'è e la penalità residua decade, così una pescata non può mai bloccare la partita. In seguito Rossi ha aggiunto a `Deck` un `Random` ricevuto dall'esterno, per rendere riproducibili i mescolamenti.

La classe `DiscardPile` mantiene la pila degli scarti, con la carta in cima come ultimo elemento della lista, e offre `takeAllButTop()` per la rigenerazione e `takeAll()` per la fine di una mano.

La costruzione dei due mazzi è affidata a `DeckFactory`, una classe di sole *factory method* statiche: `create(GameRules)` sceglie il mazzo in base al tipo indicato dalle regole (`DeckType.CLASSIC`, 108 carte, o `DeckType.NO_MERCY`, 168 carte), così che la scelta avvenga sul tipo di mazzo e non sul tipo di carta. Le quantità del mazzo No Mercy sono costanti con nome, ciascuna commentata con il totale che produce.

GameRules

Le regole di una partita sono un record, `GameRules`, e la modalità No Mercy è una configurazione diversa, non una sottoclasse di `Game`: accumulo delle penalità, soglia di eliminazione, scambio 7, rotazione 0, pesca finché giocabile, punti per vincere e tipo di mazzo. Le due factory `classic()` e `noMercy()` producono le combinazioni previste dal gioco; `withPointsToWin(int)` restituisce una copia con un diverso obiettivo di punti, usata dalla schermata *Nuova partita*. Il record contiene anche il campo `jumpIn`, previsto per la regola del "salto dentro" ma non attivo in nessuna delle due modalità.

Nella prima versione del progetto la composizione del mazzo doveva essere una mappa da tipo di carta a numero di copie; il gruppo l'ha sostituita con l'enum `DeckType`, perché creare carte da una mappa di tipi avrebbe richiesto proprio lo `switch` sul tipo che il progetto evita.

Carte No Mercy e Jolly Rimescola Mani

Ho realizzato le sette carte aggiuntive della modalità No Mercy (Pesca 4 colorato, Salta tutti, Scarta tutto, Jolly Pesca 6, Jolly Pesca 10, Jolly Cambia Giro Pesca 4, Jolly Roulette Colore), con le relative voci in `FaceSymbol`, e il mazzo da 168 carte.

A queste si aggiunge una carta ideata dal gruppo, il **Jolly Rimescola Mani** (`WildShuffleHands`), presente in una sola copia al posto di una Roulette, così il mazzo resta di 168 carte. Chi la gioca fa scambiare a caso le mani dei giocatori ancora in gioco, senza che nessuno riceva la propria, e poi sceglie il colore. Per questa carta ho scritto anche l'operazione `Game.shuffleHands()`, che distribuisce le mani secondo una permutazione senza punti fissi e notifica l'evento `HANDS_SHUFFLED`.

Partita a punti

La partita a punti è stata sviluppata a due mani. Io ho scritto la chiusura della mano con il conteggio dei punti, la raccolta delle carte a fine mano e il passaggio alla mano successiva nei controller e nelle viste; Rossi ha aggiunto il reintegro degli eliminati e i punti delle loro mani, insieme ai test.

Quando l'obiettivo è maggiore di zero, chi chiude la mano incassa il valore delle carte rimaste agli avversari: il numero per le carte numero, 20 punti per le carte azione, 50 per i jolly. Nella No Mercy le carte di chi viene eliminato tornano subito negli scarti, quindi i loro punti vengono contati al momento dell'eliminazione e assegnati a chi chiude la mano. Se nessuno ha raggiunto l'obiettivo la partita passa allo stato `HAND_OVER`; `startNextHand()` raccoglie mani e scarti nel mazzo, riammette gli eliminati, azzera il senso di gioco e distribuisce di nuovo sette carte a testa, mantenendo i punteggi.

Menu, Nuova partita e controller del menu

La finestra `MainWindow` e il menu principale (`MainMenuPanel`) sono nati con il gestore delle schermate `Navigator`, che tiene i pannelli uno sopra l'altro con un `CardLayout` e ne mostra uno alla volta, e con `ButtonFactory`, da cui si creano tutti i pulsanti. Le schermate sono i valori dell'enum `Screen`.

La schermata *Nuova partita* (`NewGamePanel`) raccoglie modalità, numero di avversari e obiettivo. Le scelte esclusive sono gruppi di pulsanti a due stati costruiti da `ChoiceGroup`, un componente generico su un tipo enum, con le etichette fornite da `ChoiceLabels`: così il controller e il modello non contengono testi. Quando l'utente preme *Inizia*, le scelte arrivano al controller come record `GameSettings`, che nel costruttore verifica che gli avversari siano fra 1 e 9. La modalità è l'enum `GameMode` (`CLASSIC`, `NO_MERCY`) e l'obiettivo l'enum `GameGoal` (`SINGLE_HAND`, `SHORT`, `LONG`), i cui valori portano con sé i punti per ciascuna modalità (una distinzione introdotta in seguito da Rossi): 100 e 500 nella Classica, 1000 e 2000 nella No Mercy, dove chi viene eliminato lascia almeno 25 carte a chi chiude la mano.

La classe `AppController` è il controller del menu e implementa `MenuActions`, l'unica interfaccia che *Nuova partita* conosce. Ricevuto il `GameSettings`, crea i nomi dei giocatori e le CPU, traduce le impostazioni in `GameRules` con il metodo statico `toRules` e affida la nuova partita al controller del tavolo, attraverso l'interfaccia `MenuView` della finestra. Se c'era una partita lasciata a metà, prima la ferma, così le sue CPU non continuano a giocare in sottofondo.

Regole del gioco

I testi delle regole delle due modalità sono file di testo semplice nelle risorse (`rules/classic.txt` e `rules/nomercy.txt`): le righe in maiuscolo sono titoli, quelle che iniziano con "- " sono voci di elenco. `RulesLoader` li legge dal classpath in UTF-8, così le lettere accentate restano corrette anche dentro il jar, e la schermata *Regole* li mostra; se un file manca, la schermata mostra un messaggio invece di andare in errore.

Vista da terminale

La vista da terminale è composta da `ConsoleMenu`, che chiede il nome del giocatore e poi, a ogni nuova partita, modalità, numero di CPU e obiettivo; da `ConsoleView`, che implementa sia `GameView` sia `GameObserver` e legge i comandi (un numero gioca la carta, `d` pesca, `k` passa, `u` *Ultima!*, `q` abbandona dopo una conferma); e da `CardFormatter`, che scrive le carte con il colore ANSI e una lettera, così da restare leggibili anche senza colori.

`GameController` è il controller della partita da terminale: dopo ogni mossa dell'utente fa giocare in sequenza tutte le CPU finché non tocca di nuovo a un umano, poi mostra lo stato e, se serve, chiede il colore o l'avversario tramite `GameView`, i cui metodi in questo caso restituiscono direttamente la scelta. Una mossa rifiutata dal modello diventa un messaggio in italiano. La classe `Main` sceglie la vista in base all'argomento `-console` o `-gui` e crea la finestra sul thread grafico di Swing con `SwingUtilities.invokeLater`; le regole si costruiscono con lo stesso `AppController.toRules` della finestra, quindi le due viste giocano esattamente le stesse partite.

Capitolo 3

Sviluppo

3.1 Testing

Il testing è stato implementato per verificare le funzionalità del sistema *Ultima Carta* i test sono stati realizzati utilizzando JUnit 5 (`junit-jupiter` 5.10.2) e si lanciano con `mvn test`. Ogni test raccoglie in sequenza i casi di una stessa regola, con un messaggio che spiega ciascuna verifica. Sono presenti i test sulle regole, sulle partite intere e sul controller, più un test per ciascuna delle classi di vista che contengono logica verificabile senza aprire una finestra. Tutti i 30 test vengono superati. Per ogni test sono riportati gli obiettivi, gli scenari, l'implementazione e i risultati attesi; la Tabella 3.1 riassume l'intera suite.

Classe di test	**Test**	**Cosa verifica**
Modello		
`ClassicRulesTest`	10	Turni, carte azione, prima carta, pesca e passo, *Ultima!*, mosse rifiutate, eventi e stato visibile
`NoMercyRulesTest`	6	Accumulo, eliminazione, pesca obbligata, nuove carte, scambio 7, rotazione 0
`PointsGameTest`	3	Punteggio di fine mano, eliminati, partite a punti intere
`FullGameTest`	2	Partite intere CPU contro CPU e riproducibilità
`CpuPlayerTest`	2	Scelte della strategia base
Controller		
`GuiGameControllerTest`	3	Controller del tavolo con vista e temporizzatore finti
Vista e audio		
`CardImageCacheTest`	1	Disegno di tutte le carte dei due mazzi
`EffectPopupTest`	1	Quando compare il popup degli effetti e con che testo
`NavigatorTest`	1	Percorso fra le schermate e pulsante *Indietro*
`AudioTest`	1	Suoni associati agli eventi della partita
	30	

Tabella 3.1: Le classi di test e il numero di test di ciascuna

3.1.1 ClassicRulesTest

Turni e carte azione

- **actionCardsChangeWhoPlaysNext**
  - **Obiettivo:** Verificare chi gioca dopo una carta numero e dopo le carte azione.
  - **Scenario:** Partite a due e tre giocatori in cui A gioca una carta numero, un Salta turno o un Inverti.
  - **Implementazione:** `PlayCard("A", 0)` e controllo del giocatore corrente, del colore attivo e del senso di gioco.
  - **Risultati attesi:** Dopo una carta numero comincia A; Salta fa perdere il turno a B; Inverti rende il senso antiorario e passa il turno a C; con due giocatori Inverti vale come Salta e il turno torna ad A.
- **drawCardsMakeTheNextPlayerDrawAndLoseTheTurn**
  - **Obiettivo:** Verificare l'effetto delle carte Pesca 2 e Jolly Pesca 4.
  - **Scenario:** Partite a tre giocatori; A gioca un Pesca 2 oppure un Jolly Pesca 4.
  - **Implementazione:** Controllo delle carte di B, dello stato e di chi deve agire, poi `ChooseColor("A", BLUE)`.
  - **Risultati attesi:** B pesca due o quattro carte e salta il turno; dopo il +4 la partita è in `AWAITING_COLOR` e il colore lo sceglie A, che l'ha giocato; scelto il blu, tocca a C.
- **aWildLetsOnlyWhoPlayedItChooseTheColor**
  - **Obiettivo:** Verificare l'attesa del colore dopo un Jolly.
  - **Scenario:** A gioca un Jolly in una partita a tre.
  - **Implementazione:** Tentativi di scelta da parte di B e di altre mosse di A con `assertThrows`, poi la scelta di A.
  - **Risultati attesi:** Il colore non lo sceglie un altro giocatore e finché non c'è nessuno può giocare o pescare; dopo la scelta il colore attivo è il blu e tocca a B.
- **theFirstCardHasItsEffect**
  - **Obiettivo:** Verificare l'effetto della prima carta scoperta, che vale come giocata dal mazziere.
  - **Scenario:** Partite che iniziano con un Jolly, un Jolly Pesca 4, un Salta turno o un Pesca 2.
  - **Implementazione:** Controllo di chi deve agire, delle carte di A e del turno dopo l'eventuale scelta del colore.
  - **Risultati attesi:** Con il Jolly il colore lo sceglie A, che poi gioca; con il +4 A pesca quattro carte, il colore lo sceglie il mazziere C e poi tocca a B; con Salta e Pesca 2 A perde il turno (e con il Pesca 2 ha nove carte).

Pesca, passo e Ultima!

- **aDrawnCardCanBePlayedRightAwayOrKept**
  - **Obiettivo:** Verificare le tre possibilità dopo una pescata.

  - **Scenario:** Una carta pescata non giocabile; una giocabile da giocare subito; una giocabile da tenere.
  - **Implementazione:** `Draw`, `PlayCard` e `Pass`, con un osservatore che registra gli eventi.
  - **Risultati attesi:** Una carta non giocabile passa il turno; una giocabile lo tiene e abilita `canPass()`, ma si può giocare solo quella e si pesca una volta sola; passando la carta resta in mano e gli eventi sono `CARDS_DRAWN` e `TURN_PASSED`.

- **anEmptyDeckIsRefilledFromTheDiscardPileOrDrawsNothing**
  - **Obiettivo:** Verificare il mazzo esaurito.
  - **Scenario:** Partite senza carte da pescare: in una gli scarti contengono una carta oltre alla cima, nell'altra solo la cima.
  - **Implementazione:** `Draw` e controllo della mano, della carta in cima e del totale delle carte.
  - **Risultati attesi:** Nel primo caso il 5 sotto la cima torna nel mazzo e viene pescato, la cima resta il 3 e il totale non cambia; nel secondo non c'è niente da pescare, la mano resta di sette carte e il turno passa.

- **lastCardMustBeCalledBeforeThePenultimateCard**
  - **Obiettivo:** Verificare la dichiarazione di *Ultima!* e la sua penalità.
  - **Scenario:** A ha due carte e gioca la penultima, con e senza dichiarazione, anche quando la penultima è un Jolly.
  - **Implementazione:** `CallLastCard` prima della carta o durante la scelta del colore; controllo della mano e degli eventi.
  - **Risultati attesi:** Chi dichiara resta con una carta, chi se ne dimentica ne ha tre; dopo un Jolly si può dichiarare mentre si sceglie il colore e la penalità arriva solo dopo la scelta; con più di due carte la dichiarazione è rifiutata; l'ultima carta chiude la partita senza penalità.

Mosse rifiutate, eventi e stato visibile

- **illegalMovesAreRejectedAndChangeNothing**
  - **Obiettivo:** Verificare che le mosse non valide vengano rifiutate senza modificare la partita.
  - **Scenario:** Partite con un solo giocatore, con undici o con nomi ripetuti; mosse prima dell'avvio, fuori turno, con una carta non giocabile e dopo la fine.
  - **Implementazione:** `assertThrows` sui costruttori e su `execute`, poi controllo di mano, cima e turno.
  - **Risultati attesi:** Ogni caso solleva l'eccezione attesa; la carta rifiutata resta in mano, la cima è ancora il 5 rosso e il turno resta ad A; a partita finita nessuno può muovere.

- **observersHearEveryMoveButNeverWhichCardsAreDrawn**
  - **Obiettivo:** Verificare gli eventi ricevuti dagli osservatori.
  - **Scenario:** Partita a tre giocatori; A gioca un Pesca 2. Un secondo osservatore viene aggiunto e poi tolto.
  - **Implementazione:** Registrazione degli eventi con `events::add` prima di `start()`.

- **Risultati attesi:** All'avvio arriva `GAME_STARTED` con la prima carta; il Pesca 2 produce `CARD_PLAYED`, `CARDS_DRAWN` (B, due carte, senza dire quali) e `PLAYER_SKIPPED`; l'osservatore tolto non riceve nulla.

- **eachPlayerSeesTheirOwnHandAndOnlyTheActingPlayerCanPass**
  - **Obiettivo:** Verificare lo stato visto da un giocatore diverso da quello di turno.
  - **Scenario:** Tocca ad A; B ha in mano una carta riconoscibile; A pesca una carta giocabile.
  - **Implementazione:** `getVisibleStateFor("B")` e `getVisibleStateFor("A")`.
  - **Risultati attesi:** Chi deve agire resta A ma B vede la propria mano; dopo la pescata può passare solo A.

3.1.2 NoMercyRulesTest

- **drawPenaltiesStackOnlyWithEqualOrHigherDrawCards**
  - **Obiettivo:** Verificare l'accumulo delle penalità.
  - **Scenario:** Tre giocatori: A ha un Pesca 2 rosso, B un Pesca 4 blu, C nessuna carta Pesca. In una seconda partita B risponde a un Pesca 4 con un Pesca 2 o con un 5.
  - **Implementazione:** Sequenza delle mosse e controllo di `pendingDraw()` dopo ognuna.
  - **Risultati attesi:** La penalità sale a 2 e poi a 6 (conta il valore, non il colore); C pesca sei carte e perde il turno. Un Pesca 2 non rilancia un Pesca 4 e con una penalità in corso le carte normali sono bloccate.

- **aReverseDrawFourAsFirstCardIsAnsweredByWhoChoosesTheColor**
  - **Obiettivo:** Verificare il Jolly Cambia Giro Pesca 4 come prima carta.
  - **Scenario:** Partita a tre giocatori che inizia con un Jolly Cambia Giro Pesca 4; il mazziere è C.
  - **Implementazione:** Controllo dello stato e di chi deve agire, poi `ChooseColor("B", BLUE)`.
  - **Risultati attesi:** Il giro si è invertito: il colore lo sceglie B, il primo dopo il mazziere nel nuovo senso, che poi deve rispondere alla penalità di quattro carte.

- **twentyFiveCardsEliminateAPlayerAndTheLastOneStandingWins**
  - **Obiettivo:** Verificare l'eliminazione a 25 carte.
  - **Scenario:** Tre giocatori: A e B giocano un Jolly Pesca 10 ciascuno e C deve pescare il totale; poi un duello a due giocatori con la stessa sequenza.
  - **Implementazione:** Controllo dello stato di C, del totale delle carte e dei turni successivi.
  - **Risultati attesi:** C, con 27 carte, è eliminato, la sua mano torna negli scarti e il giro prosegue fra A e B; nel duello resta solo B, che vince, mentre la mano vuota di A non conta come vittoria.

- **withoutAPlayableCardYouKeepDrawing**
  - **Obiettivo:** Verificare la pesca obbligata.
  - **Scenario:** Il mazzo contiene tre riempitivi e poi un 2 rosso.
  - **Implementazione:** `Draw("A")`, poi `PlayCard` della carta pescata.

  - **Risultati attesi:** A pesca quattro carte, fino a quella giocabile, resta di turno e può giocarla subito.

- **skipEveryoneDiscardAllAndTheColorRoulette**
  - **Obiettivo:** Verificare Salta tutti, Scarta tutto e il Jolly Roulette Colore.
  - **Scenario:** Quattro partite: A gioca un Salta tutti; uno Scarta tutto rosso con altre rosse in mano; uno Scarta tutto che svuota la mano; una Roulette con un riempitivo, un Jolly Pesca 6 e un 8 blu in cima al mazzo.
  - **Implementazione:** Controllo di turno, mani, cima, totale delle carte, vincitore e stato `AWAITING_ROULETTE_COLOR`.
  - **Risultati attesi:** Con Salta tutti rigioca A; Scarta tutto si porta via le altre rosse e resta in cima, e se svuota la mano vince; la Roulette la sceglie B, che scopre tre carte (il jolly non conta come blu) e perde il turno.

- **sevenSwapsAndZeroRotatesHandsOnlyInNoMercy**
  - **Obiettivo:** Verificare lo scambio del 7 e la rotazione dello 0.
  - **Scenario:** A gioca un 7 rosso e sceglie C; in un'altra partita A gioca uno 0; infine un 7 con le regole classiche.
  - **Implementazione:** `SwapHands("A", "C")` e controllo delle mani, che contengono carte riconoscibili.
  - **Risultati attesi:** A prende la mano di C e il turno passa a B; con lo 0 ogni mano passa al successivo; nella classica il 7 è un numero come gli altri.

3.1.3 PointsGameTest

- **closingAHandScoresTheOpponentsCardsAndTheGoalEndsTheGame**
  - **Obiettivo:** Verificare il punteggio di fine mano e l'obiettivo.
  - **Scenario:** Partita classica in cui A chiude la mano mentre B tiene un 8 blu e sei riempitivi da 9 (62 punti), con obiettivo 500 e poi 50.
  - **Implementazione:** Controllo di stato, punteggio, eventi e dello stato dopo `startNextHand()`.
  - **Risultati attesi:** A 500 punti la partita va in `HAND_OVER` con 62 punti ad A ed evento `HAND_WON`; la mano successiva ridistribuisce sette carte senza perderne nessuna e il punteggio resta, mentre a mano aperta `startNextHand()` è rifiutato. A 50 punti la partita è vinta: gli ultimi eventi sono `HAND_WON` e `GAME_WON`.

- **eliminatedPlayersGiveTheirPointsAndRejoinTheNextHand**
  - **Obiettivo:** Verificare i punti degli eliminati nella No Mercy.
  - **Scenario:** No Mercy a 1000 punti a due giocatori: due Jolly Pesca 10 portano A a 26 carte; poi una seconda mano chiusa da B.
  - **Implementazione:** Sequenza delle mosse, `startNextHand()` e controllo dei punteggi.
  - **Risultati attesi:** B chiude la mano e incassa $26 \cdot 9$ punti, contati prima che le carte tornassero negli scarti; nella mano successiva A rientra con sette carte; chiudendo la seconda mano B prende solo i punti delle carte che A ha adesso.
- **wholePointGamesReachTheGoalHandAfterHand**
  - **Obiettivo:** Verificare partite a punti intere su più mani.
  - **Scenario:** 60 partite classiche a 500 punti e 60 No Mercy a 1000 punti, fra 2 e 6 CPU.
  - **Implementazione:** Si gioca mano dopo mano chiamando `startNextHand()` a ogni `HAND_OVER`, controllando il totale delle carte dopo ogni mossa.
  - **Risultati attesi:** Nessuna carta persa o duplicata, nessun eliminato che resta fuori nella mano successiva; ogni partita finisce e vince chi raggiunge l'obiettivo.

3.1.4 FullGameTest

I test a mano verificano una regola per volta; per trovare i casi limite che nessuno ha previsto (mazzo esaurito, eliminazioni a catena, penalità su un giocatore appena eliminato) servono partite intere fra CPU, senza nessuna vista, controllando dopo ogni mossa un'invariante: il numero totale di carte fra mani, mazzo e scarti deve restare uguale a quello del mazzo. Che si giochino senza vista è anche la prova che il modello non dipende dall'interfaccia.

- **wholeGamesNeverLoseCardsAndEndWithOneWinner**
  - **Obiettivo:** Verificare che ogni partita finisca correttamente.
  - **Scenario:** 100 partite classiche per ciascun numero di giocatori (2, 3, 4 e 10) e 300 partite No Mercy fra 2 e 6 giocatori, ognuna con il proprio seme.
  - **Implementazione:** Dopo ogni mossa si controlla che il totale resti 108 o 168; ogni partita deve finire entro 20 000 mosse.
  - **Risultati attesi:** Nella classica il vincitore ha la mano vuota, c'è un solo evento `GAME_WON` e nessun evento di pesca dice quale carta è stata presa; nella No Mercy il vincitore non è mai un eliminato.
- **theSameSeedReplaysTheSameGame**
  - **Obiettivo:** Verificare la riproducibilità delle partite.
  - **Scenario:** Due partite a tre CPU con seme 42 e una con seme 43.
  - **Implementazione:** Confronto delle liste di eventi.
  - **Risultati attesi:** Con lo stesso seme gli eventi sono identici, con un seme diverso no.

3.1.5 CpuPlayerTest

- **playsAPlayableCardKeepingWildsForLastOrDraws**
  - **Obiettivo:** Verificare la scelta della mossa durante il turno.

  - **Scenario:** Stati visibili costruiti dal test con il 5 rosso in cima e mani diverse; uno stato dopo una pescata.
  - **Implementazione:** `decide` con la strategia base e con una strategia di prova che non gioca mai.
  - **Risultati attesi:** La CPU gioca la carta giocabile, pesca se non ne ha, tiene il jolly per ultimo e dichiara *Ultima!* prima della penultima carta; dopo una pescata gioca la carta pescata, oppure passa se la strategia la vuole tenere.

- **choosesTheColorItHasMostOfAndSwapsWithWhoHasFewestCards**
  - **Obiettivo:** Verificare le scelte richieste dalla partita.
  - **Scenario:** Stati in `AWAITING_COLOR`, `AWAITING_ROULETTE_COLOR` e `AWAITING_SWAP_TARGET`, con avversari da 5, 2 e 0 carte, quest'ultimo eliminato.
  - **Implementazione:** `decide` sui tre stati.
  - **Risultati attesi:** Dichiara il colore che ha in maggior numero, anche quando subisce la Roulette, e scambia con l'avversario che ha meno carte, mai con un eliminato.

3.1.6 GuiGameControllerTest

Il controller del tavolo si verifica senza Swing: un tavolo finto (`FakeTable`) implementa `TableView` e registra stato, scelte chieste, errori ed eventi; un temporizzatore finto (`ManualScheduler`) tiene da parte la mossa della CPU invece di aspettare, e il test la esegue quando vuole. Le mosse del giocatore umano le sceglie una `CpuPlayer` e passano dalle stesse azioni che userebbe il tavolo vero.

- **cpusMoveEveryTwoSecondsAndTheHumanWaitsForTheirTurn**
  - **Obiettivo:** Verificare l'attesa delle CPU e il turno dell'umano.
  - **Scenario:** Una partita in cui, dopo la prima mossa dell'umano, tocca a una CPU.
  - **Implementazione:** Controllo del ritardo programmato e dello stato al tavolo, poi `onDraw()` fuori turno; infine `rejectedCardMessage` su uno stato con una penalità di otto carte.
  - **Risultati attesi:** La CPU aspetta 2000 ms; il tavolo mostra sempre la mano dell'umano; fuori turno l'umano riceve "Aspetta il tuo turno." e lo stato non cambia; con la penalità il messaggio spiega che si può rilanciare solo con un +4 o più, oppure pescare otto carte.

- **pauseStopsTheCpusUntilResumeAndLeavingStopsThemForGood**
  - **Obiettivo:** Verificare pausa, ripresa e uscita dal tavolo.
  - **Scenario:** Una CPU con la mossa in attesa.
  - **Implementazione:** `onPause()`, `onResume()`, poi `onLeaveTable()` e un'ultima azione dell'umano.
  - **Risultati attesi:** In pausa la mossa è annullata, alla ripresa riparte e la pausa non cambia la partita; uscendo il timer è annullato e il tavolo non riceve più eventi.

- **wholeGamesArePlayedFromTheTableHandAfterHand**
  - **Obiettivo:** Verificare il controller su partite intere.
  - **Scenario:** 40 partite, classiche e No Mercy, a mano singola e a 200 punti, con da 1 a 5 avversari.

- **Implementazione:** Si alternano le mosse in attesa del temporizzatore, quelle dell'umano e, a mano chiusa, `onNextHand()`.
- **Risultati attesi:** Tutte le partite arrivano a un vincitore senza alcuna mossa rifiutata; con il form di fine mano aperto le CPU aspettano; almeno una volta l'umano sceglie un colore dal form e almeno una mano si chiude senza raggiungere l'obiettivo.

3.1.7 Vista e audio

- **everyCardOfBothDecksHasAFaceAndADrawing (CardImageCacheTest)**
  - **Obiettivo:** Verificare che tutte le carte si possano disegnare.
  - **Scenario:** Una faccia per ogni valore di `FaceSymbol` e tutte le carte dei due mazzi, disegnate su un'immagine in memoria.
  - **Implementazione:** `assertDoesNotThrow` sul disegno di ogni faccia con `CardRenderer`; conteggio delle facce diverse del mazzo classico.
  - **Risultati attesi:** Ogni simbolo ha un disegno (se se ne aggiunge uno senza disegnarlo il test lo segnala), le 108 carte classiche hanno 54 facce diverse e tutte le carte No Mercy si disegnano.
- **onlySpecialCardsAndEffectsShowAPopup (EffectPopupTest)**
  - **Obiettivo:** Verificare quando compare il popup degli effetti.
  - **Scenario:** Gli eventi che `Game` manda per una giocata normale, una pescata, una scelta di colore, un +2 e alcune mosse No Mercy.
  - **Implementazione:** `EffectPopup.of` sugli eventi di una mossa.
  - **Risultati attesi:** Le mosse normali e il colore scelto non mostrano nulla; il +2 mostra solo "+2" anche se chi lo subisce pesca e salta il turno; Salta tutti, Scarta tutto, Roulette e rotazione delle mani mostrano il loro titolo.
- **backNeverReturnsToATableAlreadyLeft (NavigatorTest)**
  - **Obiettivo:** Verificare il percorso fra le schermate.
  - **Scenario:** Navigazione dal menu fino al tavolo, poi a *Nuova partita* oppure al menu e alle *Opzioni*.
  - **Implementazione:** Chiamate a `show` e `back` di `Navigator`.
  - **Risultati attesi:** In entrambi i casi *Indietro* riporta al menu principale e mai al tavolo di una partita già lasciata.
- **eachGameMomentHasItsSound (AudioTest)**
  - **Obiettivo:** Verificare il collegamento fra eventi e suoni.
  - **Scenario:** Gli eventi di carta giocata, carta pescata, *Ultima!*, penalità e vittoria.
  - **Implementazione:** `GameSounds` con un lettore che raccoglie i suoni in una lista.
  - **Risultati attesi:** Ogni evento produce il proprio effetto sonoro, nell'ordine.

3.2 Metodologia di lavoro

La fase iniziale è stata svolta insieme e dedicata ai *contratti*: prima di dividersi il lavoro, il gruppo ha concordato le firme delle classi e delle interfacce che entrambi avrebbero usato, in particolare i metodi pubblici di `Game` richiamati dalle carte (`skipNext`, `reverseDirection`, `drawCards`, `setActiveColor`), l'interfaccia `Move`, gli enum e l'interfaccia della vista. Fissate le firme, il lavoro in parallelo è proceduto quasi senza sovrapposizioni.

3.2.1 Workflow

Il criterio di divisione non è stato "metà delle classi a testa", ma due aree di responsabilità distinte, bilanciate per difficoltà e non per numero di file:

- **A – "cosa esiste"** (Daniele Tontini): i dati e la configurazione, cioè carte, mazzo, pila degli scarti, composizione dei mazzi e regole; molte classi, ciascuna semplice. Per riequilibrare il carico, ad A sono andati anche il controller e la vista da terminale.
- **B – "cosa succede"** (Andrea Rossi): il motore, cioè turni, senso di gioco, stati, eventi e decisioni della CPU; poche classi, ma le più complesse del progetto. A B sono andati anche il tavolo grafico e la simulazione di partite intere.

3.2.2 Version Control

Il controllo di versione è stato gestito attraverso **Git**, con un repository privato su GitHub. Il piano iniziale prevedeva un branch e una pull request per ogni attività; il gruppo ha poi scelto di lavorare direttamente sul branch **main**, con alcune regole per evitare i conflitti:

- un `git pull` prima di iniziare a lavorare, ogni volta;
- i file condivisi (`Move`, gli enum, l'interfaccia della vista, i metodi pubblici di `Game`) si cambiano solo d'accordo, e la modifica va su `main` subito;
- commit piccoli, ciascuno con un cambiamento logico, e messaggi descrittivi in italiano;
- ognuno committa il proprio lavoro.

La cronologia conta 106 commit: 63 di Andrea Rossi e 43 di Daniele Tontini. Il progetto è costruito con **Maven**: il file `pom.xml` fissa la versione di Java (21), la dipendenza di test da JUnit 5 e i plugin per avviare il gioco (`mvn compile exec:java`) e produrre un jar eseguibile (`mvn package`).

3.2.3 Rossi

Realizzazione del motore di gioco nel package **model**, dei giocatori CPU, del controller del tavolo e della vista grafica e audio nei package **view.gui** e **view.audio**.

Il mio contributo si è concentrato su quattro aspetti principali del progetto:

- **Motore di gioco:** le classi `Game` e `Player` con la gestione dei turni, del senso di gioco, della prima carta e degli stati di attesa (`GameState`); le mosse (`Move` e i sei record); il sistema di eventi (`GameObserver`, `GameEvent`); le regole No Mercy nel motore (accumulo, eliminazione, scambio 7, rotazione 0, Roulette) con `PlayRules`; il reintegro degli eliminati e i loro punti nella partita a punti.
- **CPU:** `CpuPlayer`, l'interfaccia `CpuStrategy` e la strategia `BasicCpuStrategy`.

- **Controller del tavolo:** `GuiGameController`, `TableActions`, `TurnScheduler` con `SwingTurnScheduler` e l'interfaccia `TableView`, compresa la pausa della partita.
- **Vista grafica e audio:** il record `CardFace` con `CardRenderer`, `CardImageCache`, `PixelGlyphs` e `CardPreview`; lo stile cabinato (`ArcadeButtonUI`, `ArcadeCheckBoxUI`, `ArcadeSliderUI`, `ArcadeBackgroundPanel`, `ArcadeTitle`, `ArcadeBox`, `ScreenScale`, `FocusIndicator`); il tavolo (`TablePanel`, `SeatLayout`, `EffectPopup`, `TableTexts`, `ChoiceOverlay`, `ScoreBoard`); le opzioni (`Settings`, `OptionsPanel`, `OptionsControls`); l'audio (`AudioPlayer`, `GameSounds`, `Music`, `SoundEffect`).

Ho scritto inoltre la maggior parte dei test, la classe di supporto `GameSetup`, il `README.md` e ho curato il cambio di nome del progetto da `uno` a `lastcard`.

3.2.4 Tontini

Impostazione del progetto e realizzazione dei dati del gioco nel package **model**, della vista da terminale e del controller del menu.

Il mio contributo si è concentrato su:

- **Progetto:** il `pom.xml`, il `.gitignore` e la struttura dei tre package.
- **Carte e mazzo:** la gerarchia `Card`, `NumberCard`, `ActionCard`, `WildCard` con le carte della modalità classica e i loro effetti, le sette carte No Mercy e il Jolly Rimescola Mani con `Game.shuffleHands()`; `Deck`, `DiscardPile` e `DeckFactory` con i mazzi da 108 e 168 carte; `GameRules` e `DeckType`; la prima versione di `VisibleState` e `PlayerSummary`.
- **Partita a punti:** la chiusura della mano con il punteggio, la raccolta delle carte a fine mano e il passaggio alla mano successiva.
- **Vista da terminale:** `ConsoleView`, `ConsoleMenu`, `CardFormatter`, il controller `GameController` con la gestione delle CPU e la classe `Main`.
- **Menu della finestra:** la prima versione di `MainWindow`, `MainMenuPanel`, `Navigator` e `ButtonFactory`; la schermata *Nuova partita* (`NewGamePanel`, `ChoiceGroup`, `ChoiceLabels`) con `GameMode`, `GameGoal`, `GameSettings`, `MenuActions`, `MenuView` e `AppController`; i file delle regole con `RulesLoader`.

Le seguenti componenti sono state realizzate attraverso un lavoro collaborativo tra i due membri del team:

- **Contratti fra i package:** le firme di `Game`, `Move`, gli enum e l'interfaccia della vista (`GameView`, prima `UnoView`), concordate nella fase iniziale; in seguito le firme della fase No Mercy (`SwapHands`, lo stato `AWAITING_SWAP_TARGET`, la penalità in `VisibleState`, `askSwapTarget` e `chooseSwapTarget`), fissate prima di lavorare in parallelo.
- **Classe Game:** il motore è di Andrea Rossi, ma Daniele Tontini vi ha scritto la gestione del mazzo e degli scarti, gli effetti delle carte, il rimescolamento delle mani e la parte della partita a punti.
- **Partita a punti:** chiusura della mano e rigenerazione del mazzo da una parte, reintegro degli eliminati, obiettivi per modalità e test dall'altra.
- **Schermate della finestra:** *Nuova partita*, *Regole* e `Navigator`, impostate da Daniele Tontini e poi ridisegnate nello stile cabinato da Andrea Rossi, che ha aggiunto la cronologia delle schermate.

3.3 Note di sviluppo

3.3.1 Rossi

Durante lo sviluppo del motore, della CPU e della vista grafica ho fatto ampio uso dei costrutti introdotti nelle versioni recenti di Java, che il progetto richiede (Java 21), e dei concetti della programmazione orientata agli oggetti. Le principali tecnologie e strumenti utilizzati sono:

- **Record**: per i dati immutabili (le mosse, `GameEvent`, `CardFace`, `EffectPopup`), con costruttori compatti che validano i valori e copiano le liste con `List.copyOf`.
- **Interfacce sealed e pattern matching nello switch**: `Move` ammette solo sei implementazioni, e `Game.execute` le smista con uno `switch` esaustivo, senza ramo `default` e senza cast.

```
public void execute(Move move) {
    Objects.requireNonNull(move, "move");

    switch (move) {
        case PlayCard playCard -> executePlayCard(playCard);
        case Draw draw -> executeDraw(draw);
        case Pass pass -> executePass(pass);
        case CallLastCard callLastCard -> executeCallLastCard(callLastCard);
        case ChooseColor chooseColor -> executeColorChoice(chooseColor);
        case SwapHands swapHands -> executeSwapHands(swapHands);
    }
}
```

- **Aritmetica modulare per il giro**: il giocatore successivo si calcola con `Math.floorMod`, che resta corretto anche andando all'indietro nel senso antiorario, e salta i giocatori eliminati.

```
// L'indice del giocatore che viene dopo from nel verso indicato, saltando gli eliminati.
private int stepFrom(int from, int step) {
    int index = from;
    for (int i = 0; i < players.size(); i++) {
        index = Math.floorMod(index + step, players.size());
        if (!players.get(index).isEliminated()) {
            return index;
        }
    }
    return from;
}
```

- **Regole come dati**: il motore legge i campi di `GameRules`; il metodo `penalizeNext`, ad esempio, applica la penalità subito oppure la accumula a seconda di `drawStacking()`.

```
// Il prossimo giocatore pesca n carte e perde il turno.
// Con l'accumulo del No Mercy invece la penalità si somma e il prossimo deve rilanciarla o
      pescare il totale.
public void penalizeNext(int n) {
    Player next = getNextPlayer();
    if (rules.drawStacking()) {
        pendingDraw += n;
        // L'evento nomina chi ha giocato la carta Pesca, non chi dovrà rispondere.
        notifyObservers(GameEvent.penaltyStacked(lastPlayer.getName(), pendingDraw));
        return;
    }
    List<Card> drawn = drawCards(next, n);
    notifyObservers(GameEvent.cardsDrawn(next.getName(), drawn.size()));
    // Se è stato eliminato è già fuori dal giro: saltarlo farebbe perdere il turno anche al
      successivo.
    if (!eliminateIfOverThreshold(next) && state == GameState.IN_PROGRESS) {
```

```
            skipNext();
        }
    }
```

- **Stream, Comparator e Optional**: la strategia base sceglie la carta e l'avversario dello scambio con operazioni di filtro e di massimo o minimo su un comparatore; `Optional` rappresenta i valori che possono mancare, come la carta scelta o il vincitore.

```
@Override
public Optional<Card> choose(List<Card> playable) {
    Optional<Card> colored = playable.stream()
            .filter(card -> card.getColor() != CardColor.BLACK)
            .max(Comparator.comparingInt(Card::points));
    if (colored.isPresent()) {
        return colored;
    }
    return playable.stream().findFirst();
}

// Scambia con chi ha meno carte: è lo scambio che conviene di più.
@Override
public String chooseSwapTarget(List<PlayerSummary> opponents) {
    return opponents.stream()
            .min(Comparator.comparingInt(PlayerSummary::cardCount))
            .orElseThrow(() -> new IllegalArgumentException("There is no opponent to swap hands
      with."))
            .name();
}
```

- **Lambda e programmazione a eventi**: il controller del tavolo non contiene cicli di attesa; dopo ogni mossa decide cosa deve succedere e, se tocca a una CPU, programma la sua mossa con una lambda.

```
// Dopo ogni mossa: lo stato al tavolo, poi la scelta che il modello aspetta dall'umano,
// la prossima mossa di una CPU dopo l'attesa, oppure il vincitore.
private void afterMove() {
    if (stopped) {
        return;
    }
    VisibleState state = game.getVisibleStateFor(humanName);
    view.showState(state);
    if (game.isFinished()) {
        game.getWinner().ifPresent(winner -> view.showWinner(winner.getName()));
        return;
    }
    // Partita a punti: la mano è chiusa senza raggiungere l'obiettivo. La prossima parte da
      onNextHand().
    if (state.state() == GameState.HAND_OVER) {
        view.showHandOver(state.players());
        return;
    }

    CpuPlayer cpu = cpus.get(state.currentPlayerName());
    if (cpu != null) {
        scheduler.schedule(CPU_MOVE_DELAY_MILLIS, () -> playCpuMove(cpu));
        return;
    }
    if (state.state() == GameState.AWAITING_COLOR) {
        view.askColor(false);
    } else if (state.state() == GameState.AWAITING_ROULETTE_COLOR) {
        view.askColor(true);
    } else if (state.state() == GameState.AWAITING_SWAP_TARGET) {
```

```
            view.askSwapTarget(swapCandidates(state));
        }
    }

    // Una mossa della CPU: di solito una sola mossa, oppure Ultima! seguita dalla carta.
    // Dopo una pescata o un jolly la stessa CPU resta di turno: la sua mossa seguente arriva dopo
          un'altra attesa.
    private void playCpuMove(CpuPlayer cpu) {
        if (stopped || paused || isHandClosed()) {
            return;
        }
        for (Move move : cpu.decide(game.getVisibleState())) {
            if (isHandClosed()) {
                break;
            }
            game.execute(move);
        }
        afterMove();
    }
```

- **Timer di Swing**: `SwingTurnScheduler` usa un `javax.swing.Timer` che scatta una volta sola. Il compito viene eseguito sul thread grafico, lo stesso dei clic e dei disegni, quindi la partita non ha bisogno di sincronizzazione.

```
    private Timer timer;

    @Override
    public void schedule(int delayMillis, Runnable task) {
        cancel();
        timer = new Timer(delayMillis, e -> task.run());
        timer.setRepeats(false);
        timer.start();
    }

    @Override
    public void cancel() {
        if (timer != null) {
            timer.stop();
            timer = null;
        }
        }
    }
```

- **Mappe di riferimenti a metodo**: `CardRenderer` associa ogni simbolo al metodo che lo disegna; aggiungere una carta significa aggiungere una riga.

```
    public CardRenderer() {
        glyphs.put(FaceSymbol.NUMBER, this::number);
        glyphs.put(FaceSymbol.DRAW_TWO, this::drawTwo);
        glyphs.put(FaceSymbol.REVERSE, this::reverse);
        glyphs.put(FaceSymbol.SKIP, this::skip);
        glyphs.put(FaceSymbol.WILD, this::wild);
        glyphs.put(FaceSymbol.WILD_DRAW_FOUR, this::wildDrawFour);

        // No Mercy
        glyphs.put(FaceSymbol.DRAW_FOUR, this::colorDrawFour);
        glyphs.put(FaceSymbol.SKIP_EVERYONE, this::skipEveryone);
        glyphs.put(FaceSymbol.DISCARD_ALL, this::discardAll);
        glyphs.put(FaceSymbol.WILD_DRAW_SIX, this::wildDrawSix);
        glyphs.put(FaceSymbol.WILD_DRAW_TEN, this::wildDrawTen);
        glyphs.put(FaceSymbol.WILD_REVERSE_DRAW_FOUR, this::wildReverseDrawFour);
        glyphs.put(FaceSymbol.WILD_COLOR_ROULETTE, this::colorRoulette);
```

```
        glyphs.put(FaceSymbol.WILD_SHUFFLE_HANDS, this::shuffleHands);

        cornerGlyphs.put(FaceSymbol.REVERSE, this::reverseCorners);
        cornerGlyphs.put(FaceSymbol.SKIP, this::skipCorners);
        cornerGlyphs.put(FaceSymbol.WILD, this::wildCorners);
        cornerGlyphs.put(FaceSymbol.SKIP_EVERYONE, this::skipEveryoneCorners);
        cornerGlyphs.put(FaceSymbol.DISCARD_ALL, this::discardAllCorners);
        cornerGlyphs.put(FaceSymbol.WILD_COLOR_ROULETTE, this::colorRouletteCorners);
        cornerGlyphs.put(FaceSymbol.WILD_SHUFFLE_HANDS, this::wildCorners);
    }
```

- **Cache con chiave record**: un record ha `equals` e `hashCode` per valore, quindi è una chiave naturale per una `HashMap`; `computeIfAbsent` disegna l'immagine solo la prima volta, alla risoluzione reale dello schermo.

```
    /** Una voce: cosa c'è disegnato e a che dimensione. {@code face} è null per il dorso. */
    private record Key(CardFace face, int pixelWidth, boolean playable) {
    }

    private void paint(Graphics2D g, Key key, int x, int y, int width) {
        BufferedImage image = images.computeIfAbsent(key, this::render);
        Graphics2D copy = (Graphics2D) g.create();
        try {
            // l'immagine ha già la risoluzione dello schermo: va solo riportata alla dimensione
           logica
            copy.setRenderingHint(RenderingHints.KEY_INTERPOLATION,
           RenderingHints.VALUE_INTERPOLATION_BILINEAR);
            copy.drawImage(image, x, y, width, CardRenderer.heightFor(width), null);
        } finally {
            copy.dispose();
        }
    }

    /**
     * Larghezza reale in pixel. Con lo schermo scalato da Windows (125%, 150%)
     * una carta larga 130 occupa 163 o 195 pixel veri: si disegna a quella
     * risoluzione, altrimenti l'immagine ingrandita viene sfocata.
     */
    private static int pixelWidth(Graphics2D g, int width) {
        AffineTransform t = g.getTransform();
        double scale = Math.max(1.0, Math.hypot(t.getScaleX(), t.getShearY()));
        return (int) Math.round(width * scale);
    }
```

- **Java 2D e trasformazione inversa per i clic**: tutto il tavolo è disegnato con `Graphics2D`; le carte della mano sono ruotate con `AffineTransform`, e per sapere quale carta è sotto il puntatore il punto viene riportato nel sistema di riferimento di ciascuna carta, partendo da quella disegnata per ultima.

```
    // La carta sotto il punto: si guardano dall'ultima disegnata, che sta sopra le altre.
    private int cardAt(int x, int y) {
        for (int i = handCards.size() - 1; i >= 0; i--) {
            HandCard card = handCards.get(i);
            try {
                Point2D local = card.transform().inverseTransform(new Point2D.Double(x, y), null);
                if (card.bounds().contains(local)) {
                    return card.index();
                }
            } catch (NoninvertibleTransformException e) {
                // una rotazione è sempre invertibile: non succede
            }
        }
```

```
        return -1;
    }
```

- **Switch come espressione**: `EffectPopup` traduce gli eventi di una mossa nel testo del popup.

```
    private static String effect(GameEvent event, String viewer, boolean someoneDrew) {
        boolean me = event.playerName() != null && event.playerName().equals(viewer);
        return switch (event.type()) {
            // anche la prima carta scoperta ha il suo effetto, come se l'avesse giocata il mazziere
            case CARD_PLAYED, GAME_STARTED -> card(event.card());
            case DIRECTION_REVERSED -> "CAMBIO GIRO";
            case PLAYER_SKIPPED -> someoneDrew ? null : "SALTO DEL TURNO";
            case ALL_SKIPPED -> "SALTA TUTTI";
            case PENALTY_STACKED -> TableTexts.upper("Penalità: " + TableTexts.cards(event.count()));
            case HANDS_SWAPPED -> "SCAMBIO DI MANI";
            case HANDS_ROTATED -> "ROTAZIONE DELLE MANI";
            case HANDS_SHUFFLED -> "MANI RIMESCOLATE";
            case PLAYER_ELIMINATED -> me ? "SEI ELIMINATO" : event.playerName().toUpperCase() + "
          ELIMINATO";
            default -> null;
        };
    }
```

- **Cronologia delle schermate**: `Navigator.show` aggiorna il percorso fatto, accorciandolo quando si torna a una schermata già visitata.

```
    public void show(Screen screen) {
        Objects.requireNonNull(screen, "screen");
        if (!registered.contains(screen)) {
            throw new IllegalArgumentException("Screen not registered: " + screen);
        }
        int earlier = history.indexOf(screen);
        if (earlier >= 0) {
            history.subList(earlier, history.size()).clear();
        } else if (current != null && current != screen) {
            history.add(current);
        }
        current = screen;
        layout.show(container, screen.name());
    }
```

- **Audio ed EnumMap**: `javax.sound.sampled.Clip` per gli effetti e la musica, con la dissolvenza applicata ai campioni PCM a 16 bit; `GameSounds` associa gli eventi ai suoni con una `EnumMap`.

```
    // Gli eventi che hanno un suono; gli altri passano in silenzio.
    private static final Map<GameEvent.Type, SoundEffect> SOUNDS = new
          EnumMap<>(GameEvent.Type.class);

    static {
        SOUNDS.put(GameEvent.Type.CARD_PLAYED, SoundEffect.CARD_PLAYED);
        SOUNDS.put(GameEvent.Type.CARDS_DRAWN, SoundEffect.CARD_DRAWN);
        SOUNDS.put(GameEvent.Type.LAST_CARD_CALLED, SoundEffect.LAST_CARD);
        SOUNDS.put(GameEvent.Type.LAST_CARD_PENALTY, SoundEffect.PENALTY);
        SOUNDS.put(GameEvent.Type.GAME_WON, SoundEffect.VICTORY);
    }
```

- **Preferences**: `Settings` salva le opzioni con `java.util.prefs.Preferences`, senza file di configurazione da gestire a mano.
- **Oggetti finti nei test**: grazie alle interfacce `TableView` e `TurnScheduler`, i test del controller del tavolo usano un tavolo che registra ciò che gli viene chiesto e un temporizzatore che esegue la mossa in attesa solo quando lo decide il test, senza aprire finestre e senza attese reali.

3.3.2 Tontini

- **Polimorfismo al posto dei controlli di tipo**: l'effetto di ogni carta è nella sua classe. Il Jolly Cambia Giro Pesca 4, ad esempio, compone tre operazioni elementari di `Game`; con `flipDirection()` invece di `reverseDirection()` la penalità non torna a chi ha giocato la carta quando i giocatori sono due.

```
@Override
public void applyEffect(Game game) {
    game.flipDirection();
    game.penalizeNext(drawValue());
    game.awaitColorChoice();
}
```

- **equals e hashCode**: due carte sono uguali se hanno la stessa classe, lo stesso colore e lo stesso simbolo, così si possono confrontare e cercare nelle liste.

```
/*due carte sono uguali se hanno la stessa classe,colore,simbolo */
@Override
public boolean equals(Object o) {
    if (this == o) {
        return true;
    }
    if (o == null || getClass() != o.getClass()) {
        return false;
    }
    Card other = (Card) o;
    return color == other.color && symbol().equals(other.symbol());
}

@Override
public int hashCode() {
    return Objects.hash(getClass(), color, symbol());
}
```

- **Factory method statiche**: `GameRules.classic()` e `GameRules.noMercy()` danno un nome alle due configurazioni, e `DeckFactory` costruisce il mazzo corrispondente.

```
/** Regole classiche, una mano sola. */
public static GameRules classic() {
    return new GameRules(false, 0, false, false, false, false, 0, DeckType.CLASSIC);
}

/** No Mercy: accumulo, eliminazione a 25 carte, scambio 7, rotazione 0, pesca finché giocabile.
      Una mano sola. */
public static GameRules noMercy() {
    return new GameRules(true, 25, true, true, false, true, 0, DeckType.NO_MERCY);
}
```

- **Rigenerazione e riuso delle carte**: il mazzo si rigenera dagli scarti tranne la cima e, nella partita a punti, a fine mano tutte le carte tornano nel mazzo, comprese quelle degli eliminati.

```
// Rimette nel mazzo gli scarti, tranne la carta in cima, e rimescola.
public void refillFrom(DiscardPile discardPile) {
    cards.addAll(discardPile.takeAllButTop());
    shuffle();
}

// Le mani di tutti e gli scarti, cima compresa, tornano nel mazzo. Il punteggio resta.
// Gli eliminati della mano chiusa rientrano: l'eliminazione vale per una mano sola.
private void collectCards() {
    for (Player player : players) {
```

```
            deck.addCards(player.takeHand());
            player.resetLastCardCall();
            player.rejoin();
        }
        deck.addCards(discardPile.takeAll());
    }
```

- **Permutazioni senza punti fissi**: il Jolly Rimescola Mani distribuisce le mani con uno *scompiglio*, una permutazione in cui nessuno riceve la propria mano, usando il `Random` della partita così che anche questa scelta casuale sia riproducibile.

```
    // Una permutazione a caso di 0..size-1 in cui nessuno resta al suo posto: la mano i va al
        giocatore receivers[i].
    // Si rimescola finché qualcuno riceverebbe la propria mano; servono in media meno di tre
        tentativi.
    private List<Integer> derangement(int size) {
        List<Integer> receivers = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            receivers.add(i);
        }
        boolean someoneKeepsOwn = size > 1;
        while (someoneKeepsOwn) {
            Collections.shuffle(receivers, random);
            someoneKeepsOwn = false;
            for (int i = 0; i < size; i++) {
                if (receivers.get(i) == i) {
                    someoneKeepsOwn = true;
                }
            }
        }
        return receivers;
    }
```

- **Map immutabili**: `CardFormatter` traduce i simboli interni delle carte nei testi del terminale con `Map.ofEntries`, perché `Map.of` accetta al massimo dieci coppie.

```
    // Testo mostrato per i simboli interni delle carte; numeri e +2 restano come sono.
    // Map.ofEntries perché Map.of accetta al massimo dieci coppie.
    private static final Map<String, String> LABELS = Map.ofEntries(
            Map.entry("SKIP", "Salta"),
            Map.entry("REVERSE", "Inverti"),
            Map.entry("WILD", "Jolly"),
            Map.entry("+4", "Jolly +4"),
            // No Mercy
            Map.entry("DRAW_FOUR", "+4"),
            Map.entry("WILD_DRAW_SIX", "Jolly +6"),
            Map.entry("WILD_DRAW_TEN", "Jolly +10"),
            Map.entry("WILD_REVERSE_DRAW_FOUR", "Jolly Giro +4"),
            Map.entry("SKIP_EVERYONE", "Salta tutti"),
            Map.entry("DISCARD_ALL", "Scarta tutto"),
            Map.entry("WILD_COLOR_ROULETTE", "Jolly Roulette"),
            Map.entry("WILD_SHUFFLE_HANDS", "Jolly Rimescola Mani"));

    /** Per esempio [R 5] in rosso, [B Salta] in blu, [Jolly +4]. */
    public String format(Card card) {
        String label = LABELS.getOrDefault(card.symbol(), card.symbol());
        String text = card.getColor() == CardColor.BLACK
                ? "[" + label + "]"
                : "[" + initial(card.getColor()) + " " + label + "]";
        return paint(card.getColor(), text);
    }
```

- **Lettura dei comandi**: `ConsoleView` traduce ogni comando digitato in una chiamata al controller, e un input non valido diventa un messaggio invece di un'eccezione.

```
// Traduce un comando in un'azione del controller: numero = gioca, d = pesca, k = passa, u =
      Ultima!, q = esci
private void handleCommand(String command, GameController controller) {
    if (command.equals("q")) {
        if (confirmLeave()) {
            System.out.println("Partita abbandonata.");
            controller.onLeaveGame();
        }
    } else if (command.equals("d")) {
        controller.onDraw();
    } else if (command.equals("k")) {
        controller.onPass();
    } else if (command.equals("u")) {
        controller.onLastCardCalled();
    } else {
        try {
            controller.onCardSelected(Integer.parseInt(command));
        } catch (NumberFormatException e) {
            showError("Comando non riconosciuto. Usa un numero, d, k, u oppure q.");
        }
    }
}
```

- **Collezioni principali**:
  - `List<Card>`: mazzo, scarti e mani, con la cima in fondo alla lista;
  - `Map<String, String>` e `Map<CardColor, String>`: testi e colori ANSI del terminale;
  - `enum`: `CardColor`, `DeckType`, `GameMode` e `GameGoal`.

Capitolo 4

Sorgenti

4.1 Link Github

https://github.com/danieletontini/LastCard

4.2 Compilazione e avvio

Il progetto richiede **Java 21** o successivo e **Maven 3**; non ha dipendenze esterne oltre a JUnit 5 per i test. Dalla cartella del progetto:

```
mvn compile exec:java                           # finestra grafica
mvn compile exec:java -Dexec.args="--console"   # terminale
mvn test                                        # test automatici
mvn package                                     # crea target/lastcard.jar
java -jar target/lastcard.jar [--console]
```

Per sviluppare le carte è disponibile la classe `CardPreview`, che produce un'immagine con tutte le facce, `anteprima-carte.png` (Figura 2.7):

```
mvn -q compile exec:java -Dexec.mainClass=lastcard.view.gui.CardPreview
```

Nella versione da terminale si gioca digitando un numero per giocare la carta in quella posizione, `d` per pescare, `k` per tenere la carta pescata e passare, `u` per dichiarare *Ultima!* e `q` per abbandonare la partita. Un estratto di una partita (i caratteri grafici del terminale sono semplificati):

```
-- Turno di Andrea --
In cima: [B 7]   Colore attivo: Blu
Avversari: CPU 1 6
La tua mano: 0:[R 5]  1:[V +2]  2:[G 4]  3:[V Inverti]  4:[R 2]  5:[V 7]  6:[G 1]
Comandi: numero = gioca la carta, d = pesca, u = Ultima!, q = esci
> d
>> [Andrea] pesca 1 carta
>> [CPU 1] gioca [V 7]
```

4.3 Crediti

- Carattere *Press Start 2P*, distribuito con licenza SIL Open Font License (`src/main/resources/font/OFL-PressStart2P.txt`).
- Musica ed effetti sonori: brani da Pixabay con la relativa licenza, oltre a suoni realizzati dal gruppo; l'elenco completo è in `src/main/resources/audio/CREDITS.txt`.
- Immagini di sfondo: elencate in `src/main/resources/images/CREDITS.txt`.